Embedded Artificial Intelligence

Cours et méthodes : cours 01 à 05, réseaux de neurones, TP 1 à 4

BMEVIMIMA22, Béla Pataki, automne 2026. Rédigé le 2026-09-28, complété le 2026-10-17.

À retenir

Midterm	50 points, le 2026-10-29 de 18h à 20h
Programme	jusqu'aux réseaux de neurones (cours 6) inclus
Rattrapage	le 2026-11-09 de 18h à 20h
Signature	40 % au midterm, soit 20 points sur 50 ; sans elle, pas d'examen
Format	hybride, QCM et questions ouvertes (enseignant, 2026-09-24) ; VRAI/FAUX à +1 et −1 dans les séries d'entraînement 2026
Quiz en cours	1 à 4 par cours, 1/3 de point par bonne réponse ; plafond **À VÉRIFIER**
Examen final	écrit, 50 points, slides et lectures complémentaires
Note	examen +0,2× midterm (max 10) + quiz
Réussite	40 % de (50 + 10), soit 24 points

Source : fiche `CLAUDE.md` de la matière ; `01_Intro`, slide 3

Exemple. Midterm à 30/50 : 0,2×30 = 6 points ; sans quiz, il reste 24−6 = 18 points sur 50 à obtenir à l'examen final.

Piège

Le midterm du 29 octobre est le premier essai ; le rattrapage du 9 novembre sert en cas d'échec ou d'absence. Qu'il compte aussi comme midterm dans la note finale : confirmation écrite **À VÉRIFIER**. VRAI/FAUX à −1 par erreur : ne répondre qu'aux items sûrs.

Périmètre. Tout le programme du midterm, cours 1 à 6. Supports : `01_Intro` (36 slides), `02_Challenges` (34 pages), `03_Classification` (52 pages), `04_Clustering` (51 pages), `05_UseCase` (18 pages), notebooks des TP 1 et 2 avec solutions, scripts MATLAB du TP 3, slides du TP 4 (`NN2a`, `NN4a`) et du dépôt `Fundamentals-of-Neural-Networks` (`NN1`, `NN8`). Concepts 1 à 19 :

cours 1 et 2, TP 1 et 2 ; 20 à 26 : classification ; 27 à 34 : clustering, PCA et TP 3 ; 35 : cas d'usage du cours 5 ; 36 à 43 : réseaux de neurones.

> **Piège**
>
> **Slides du cours 6 absentes du dépôt.** La partie réseaux de neurones (concepts 36 à 43) s'appuie sur les supports du TP 4, pas sur les slides du cours magistral : notations et convention d'erreur à confirmer (concept 36).

Sources. Slide (`01_Intro`, slides PowerPoint du TP 4), page du PDF (`02_Challenges` à `05_UseCase`, le numéro de page est celui de la slide), cellule (notebooks), fichier (scripts MATLAB). [rappel général] marque une explication générale absente des supports. Calculs vérifiés par `calc_embai.py` (concepts 1 à 19) et `calc_embai_s36.py` (concepts 20 à 43). Exercices du même programme : `entrainement-embai.pdf` et `fiche-midterm-practice.pdf` (séries d'entraînement 2026).

Table des matières

1 Systèmes embarqués et informatique généraliste 7

1.1 Trois exemples, trois profils de contraintes 7

2 Qu'est-ce que l'IA ? 8

2.1 Quatre approches (slide 14) 8

2.2 Repères historiques (slides 15 et 16) 9

2.3 Domaines d'application (slide 17) 9

3 IA, ML, DL, IA générative ; modèle ou données 10

3.1 Méthode fondée sur un modèle ou sur les données (slides 20 et 21) 10

3.2 La qualité des données (slides 22 et 23) 10

4 Ordres de grandeur : le coût des grands modèles 11

5 IA embarquée : AIoT, TinyML, EdgeAI 12

6 Complexité algorithmique et notation O 13

6.1 Le problème des pièces (slides 32 et 33) 14

6.2 Algorithmes de tri (slide 34) 15

7 Apprentissage automatique : les principes 15

8 Défis 1 à 5 : entraînement, taille, énergie, vitesse, encombrement 17

8.1 Défi 1, temps d'entraînement (p. 10 et 11) 17

8.2 Défi 2, taille du modèle (p. 12 à 14) 18

8.3 Défi 3, efficacité énergétique (p. 15) 18

8.4 Défi 4, vitesse d'inférence (p. 16) 19

8.5 Défi 5, taille et poids physiques (p. 17) 19

9 Défis 6 à 8 : sûreté, confiance, gouvernance **19**
9.1 Défi 6, sûreté (p. 18) 19
9.2 Défi 7, confiance (p. 19 à 22) 19
9.3 Défi 8, gouvernance (p. 23) 20

10 Où calculer : cloud ou local **21**

11 Où part l'énergie **22**

12 Matériel, premières applications, capteurs intelligents **23**

13 Représentation des nombres et quantification **24**
13.1 Taille mémoire d'un modèle 25

14 TP 1 : Python **25**
14.1 Exercices du TP et solutions 26

15 TP 1 : NumPy et Matplotlib **27**
15.1 Matplotlib 28

16 TP 2 : régression linéaire, solution analytique **29**

17 TP 2 : régression polynomiale et régularisation **31**

18 TP 2 : jeu de données réel et RMSE **32**

19 TP 2 : régression logistique **33**

20 Classification : le problème **36**

21 Plus proches voisins (kNN) **36**

22 Arbres de décision **38**

23 Entropie et gain d'information **39**

24 Problèmes de l'apprentissage supervisé **41**

25 Matrice de confusion et métriques **42**

26 Courbe ROC et AUC **43**

27 Clustering : objectif et préparation **45**

28 Distances **45**

29 K-means **46**

29.1 Yinyang k-means, pour l'embarqué (p. 10 à 14) 47

30 Clustering hiérarchique **48**

31 DBSCAN et données manquantes **49**

32 Malédiction de la dimension **50**

33 Analyse en composantes principales (PCA) **51**

34 TP 3 : PCA par SVD, compression d'image **52**

35 Cours 5 : détection de chute sur un système portable **54**

36 Réseaux de neurones : sources utilisées **56**

37 Le neurone **56**

38 Fonctions d'activation **57**

39 Architectures et règles d'apprentissage **58**

40 Rétropropagation **60**

41 Méthode : un pas d'apprentissage (type quiz 06.3) **61**

42 Entraîner en pratique **62**

43 Généralisation, bruit, early stopping **63**

44 Ce qui reste au programme **64**

45 Méthode d'examen **65**

46 Formulaire **69**

1. Systèmes embarqués et informatique généraliste

Source : `01_Intro`, slides 6 à 9

Critère	Ordinateur généraliste	Système embarqué
But	large éventail de tâches (PC, serveurs)	optimisé pour une tâche précise, donc plus efficace pour elle
Contraintes	peu contraint	mémoire, puissance de calcul, temps de réponse, consommation limités : conception matérielle et logicielle spécialisée
Interface	riche, interaction directe	souvent aucune, ou minimale
Durée de vie	remplacé en quelques années	souvent dix ans ou plus (aéronautique, automobile)
Logiciel	Windows, macOS, Linux	OS temps réel, ou aucun OS

Trois exemples, trois profils de contraintes

Exemple	Important	Non critique
Maison connectée (slide 7)	communication, fusion d'informations, consommation (d'une partie au moins) des capteurs	mémoire, puissance de calcul, temps de réponse
Voiture autonome (slide 8)	puissance de calcul, temps de réponse, communication, fusion d'informations	mémoire, consommation des capteurs, poids
Surveillance environnementale (slide 9)	communication, fusion, consommation des appareils (batterie, « !!! »), petit, léger et robuste	mémoire, puissance de calcul, temps de réponse

Capteurs cités. Maison : portes et fenêtres, température, consommation électrique, mouvement, fumée, fuite d'eau ; commande à distance (mobile, voix) et minuterie pour économiser l'énergie. Voiture : caméras, LIDAR, ultrasons, radar, centrale inertielle, GPS ; commande automatique ou humaine, vocale

ou physique. Forêt : de simples capteurs audio, un système de détection et une alarme contre les motards qui traversent une forêt protégée (sécurité, bruit, dégâts).

> **Piège**
>
> Aucun des trois n'a « tout » critique. La voiture ne se soucie pas de la consommation des capteurs ; le capteur en forêt ne se soucie pas du temps de réponse. Une question type : « quelles contraintes sont critiques pour telle application ? ». Raisonner depuis l'alimentation (secteur, batterie de voiture, pile) et depuis la gravité d'une réponse tardive.

2. Qu'est-ce que l'IA ?

Source : `01_Intro`, slides 10 à 17

L'intelligence est une notion floue ; les chercheurs en IA utilisent surtout la **rationalité** : choisir la meilleure action pour atteindre un but, selon des critères à optimiser et avec les ressources disponibles (slide 11). Définition du Parlement européen (slide 13) : capacité d'une machine à montrer des aptitudes humaines comme le raisonnement, l'apprentissage, la planification et la créativité.

Test de Turing (1950, slide 13) : un juge humain dialogue avec une personne et une machine ; s'il ne sait pas dire laquelle est la machine, celle-ci réussit le test.

Quatre approches (slide 14)

	Comme un humain	Rationnellement
Penser	penser comme un humain	**penser rationnellement**
Agir	agir comme un humain	**agir rationnellement**

La slide entoure la colonne « rationnellement » : « humane manner ≠ rational ». Approche d'ingénieur : un but fixé d'avance, atteint avec le moins d'effort, de temps ou d'énergie. Les buts s'expriment en utilité des résultats ; le monde est incertain, donc on utilise l'utilité espérée ; être rationnel, c'est **maximiser l'utilité espérée**. La voie humaine n'est pas forcément la

meilleure : les avions à ailes battantes ont échoué, la roue et la route vont plus vite que les jambes.

Repères historiques (slides 15 et 16)

1950	Alan Turing, test de Turing
1956	atelier de Dartmouth : naissance de l'IA
1966	premier chatbot, ELIZA
1970 à 1980	hiver de l'IA après le premier boom, recherche en pause
1997	Deep Blue (IBM) bat Garry Kasparov aux échecs
2011	Watson (IBM) gagne à Jeopardy ; Siri sur l'iPhone 4S
2012	AlexNet, CNN de reconnaissance d'images
2014	Amazon Alexa ; le chatbot Eugene Goostman passe le test de Turing
2016	Tay suspendu après 16 heures pour ses messages racistes et sexistes
2017	AlphaGo bat Ke Jie au go
2018	premier GPT ; lignes directrices éthiques de l'UE pour une IA digne de confiance
2020	GPT-3

Domaines d'application (slide 17)

La slide encadre trois applications comme **typiquement embarquées** : conduite autonome (Tesla), prédiction du trafic, surveillance environnementale. Trois autres sont **généralistes ou embarquées** : maintenance prédictive, reconnaissance de formes et d'images, reconnaissance vocale. Les autres (recommandation de produits, trading, assistant personnel, filtrage de spam et de malware, découverte de médicaments et diagnostic, programmation) ne sont pas encadrées. Slide 19 : parmi les métiers qu'une IA pourrait faire mieux que l'humain (étude BBC), la slide marque comme embarqués « plier le linge », « chauffeur routier » et « chirurgien ».

3. IA, ML, DL, IA générative ; modèle ou données

Source : `01_Intro`, slides 18 à 23

> **À retenir**
>
> - **IA** : techniques qui permettent à un ordinateur de résoudre des problèmes « difficiles », de s'adapter, d'apprendre, d'explorer.
> - **Apprentissage automatique (ML)** : sous-ensemble de l'IA ; des algorithmes prédisent à partir de données. L'information est donnée sous forme d'*exemples*, pas de lois, de règles ou de formules.
> - **Apprentissage profond (DL)** : sous-ensemble du ML ; réseaux de neurones à nombreuses couches qui extraient eux-mêmes les caractéristiques des données brutes.
> - **IA générative** : sous-ensemble du DL qui produit du contenu (texte, images, code), entraîné sur de très grands jeux de données.

Schéma de la slide 18. Quatre ensembles emboîtés : IA ⊃ ML ⊃ DL ⊃ IA générative. Une ellipse « Embedded AI » traverse les cercles IA, ML et DL : l'IA embarquée n'est pas une couche de plus, elle recoupe plusieurs niveaux.

Méthode fondée sur un modèle ou sur les données (slides 20 et 21)

Avec modèle. On connaît la structure : le circuit est linéaire, sa fonction de transfert a une forme connue, par exemple un facteur $1/(1 + j\omega R_3 C)$. On mesure V_1 et V_2 à plusieurs fréquences et on calcule R_1 à R_4, L et C. Questions : le modèle est-il juste ? comment obtenir les valeurs ? précision et bruit de mesure.

Sans modèle (ML). La boîte est noire : $V_2 = f(V_1(\omega))$. On n'a que des données (un tableau de V_1, ω, V_2 mesurés). Questions : combien de données faut-il ? comment passer des données à la prédiction ? précision et bruit de mesure. Même but dans les deux cas : prédire V_2 pour tout V_1 et tout ω.

La qualité des données (slides 22 et 23)

Le ML apprend sur des exemples « sans être explicitement programmé » : l'algorithme d'entraînement est général, pas propre au problème. Il faut donc des données pertinentes, non biaisées et peu bruitées ; nettoyer les données

(réduction du bruit, détection et traitement des valeurs aberrantes) est capital, même si le cours ne le traite pas faute de temps. Chiffre cité (Gartner) : d'ici 2027, les organisations qui soignent la sémantique de leurs données pourraient gagner jusqu'à 80 % de précision et réduire leurs coûts de 60 %.

> **Piège**
>
> « Garbage in, garbage out » (slide 23) : le meilleur algorithme du monde, avec des données non pertinentes, bruitées ou biaisées, produit un résultat sans valeur.

4. Ordres de grandeur : le coût des grands modèles

Source : `01_Intro`, slides 24 à 26

Modèle	Année	Paramètres (slide 24)
GPT-1	2018	117 millions
GPT-2	2019	1,5 milliard
GPT-3	2020	175 milliards
GPT-3.5	2022	175 milliards
GPT-4.5	2023	1,7 billion ($1{,}7 \cdot 10^{12}$)

Entraînement (slide 25)	GPT-3	GPT-4
Durée	34 jours	100 jours
Énergie	1 287 000 kWh	62 318 800 kWh
Coût	168 597 $	8 163 763 $

> **Calculs de la slide 25**
>
> GPT-4 : 62 318 800 kWh $= 6{,}23 \cdot 10^{10}$ Wh, à comparer aux $\approx 6{,}1 \cdot 10^{10}$ Wh que consomme la Hongrie en une demi-journée : ratio 1,02. GPT-4 contre GPT-3 : 62 318 800/1 287 000 $\approx$ 48 fois plus d'énergie. Prix implicite : 168 597/1 287 000 = 0,131 $ par kWh, le tarif indiqué en bas de la slide.

Calcul de la slide 26 : l'inférence

$2 \cdot 10^9$ requêtes par jour à 0,34 Wh chacune : $2 \cdot 10^9 \times 0{,}34 = 0{,}68 \cdot 10^9$ Wh par jour. La Hongrie consomme $0{,}85 \cdot 10^9$ Wh en 10 minutes : ChatGPT consomme chaque jour l'équivalent de $10 \times 0{,}68/0{,}85 = 8$ minutes de consommation du pays.

Répartition de l'énergie d'une requête (camembert de la slide 26) : 75 % calcul, 10 % refroidissement, 10 % gestion des données, 5 % infrastructure.

À retenir

Message du cours : l'IA « sur ordinateur » coûte énormément d'énergie, à l'entraînement comme à l'inférence. En embarqué, l'énergie est limitée (batterie) : il faut d'autres modèles et d'autres techniques.

5. IA embarquée : AIoT, TinyML, EdgeAI

Source : `01_Intro`, slides 27 à 30

Mots-clés du domaine (slide 27) : AIoT, TinyML, EdgeAI, MLSensors. Algorithmes très optimisés et à faible empreinte, y compris pour l'audio et la vidéo. Applications « toujours actives » : il faut minimiser la consommation, et elles posent un problème de vie privée. Matériel dédié basse consommation : microcontrôleurs et DSP ultra basse consommation, couplés étroitement aux capteurs, sous-systèmes d'appareils plus complexes.

Techniques TinyML annoncées pour plus tard (slide 28) : **quantification** (réduire la précision numérique des poids), **élagage** (*pruning*, supprimer des poids ou neurones inutiles), algorithmes spécialisés. Chaîne de la figure : entraînement, distillation, quantification, encodage, compilation ; le modèle TensorFlow est converti au format `.tflite`, puis en tableau C chargé sur le microcontrôleur.

Avantages d'EdgeAI (slide 29) : latence réduite, confidentialité (pas d'envoi vers un serveur), économie de bande passante, résilience (fonctionne sans connexion). Exemple de matériel : kit NVIDIA Jetson Nano (CPU ARM Cortex-A57, GPU Maxwell, 4 Go de RAM).

	TinyML (slide 28)	EdgeAI (slide 29)
Idée	ML sur matériel très contraint : microcontrôleurs	exécuter l'IA sur un appareil local, près de la source des données
Cibles	IoT, objets portés, edge computing	IoT, smartphones, serveurs de périphérie, puces IA
Applications	objets connectés, wearables	véhicules autonomes, caméras, drones, capteurs industriels
Outils	TensorFlow Lite for Microcontrollers	TensorFlow Lite, ONNX Runtime, NVIDIA Jetson
Défis	mémoire, calcul, énergie	ressources limitées face au cloud ; compression et optimisation du modèle

Ce que le cours va traiter (slide 30) : les ressources limitent la complexité de l'algorithme (consommation, temps de réponse, mémoire) ; on verra des méthodes d'IA importantes (pas toutes), les options matérielles et la communication. Les méthodes et le matériel changent vite, mais les algorithmes de base et les grandes tendances restent.

6. Complexité algorithmique et notation O

Source : `01_Intro`, slides 31 à 35

Complexité en temps : nombre d'exécutions des instructions, pas la durée mesurée (qui dépend du compilateur et du processeur). **Complexité en espace** : mémoire totale nécessaire. Les deux s'expriment en fonction de la taille N de l'entrée ; on distingue cas moyen et pire cas (slides 31 et 34).

> **À retenir**
>
> **Définition (slide 31).** $T(N)$ est $O(F(N))$ s'il existe des constantes positives c et N_0 telles que $T(N) \leq c\,F(N)$ pour tout $N > N_0$: une borne supérieure pour les grands N.
> O : borne supérieure ; Ω : borne inférieure ; Θ : ordre exact (slide 35).

Prouver que $3N^2 + 5N + 2$ est $O(N^2)$ (méthode générale)

On choisit $c = 4$. Il faut $3N^2 + 5N + 2 \leq 4N^2$, soit $N^2 - 5N - 2 \geq 0$. La racine positive vaut $(5+\sqrt{33})/2 \approx 5{,}37$: l'inégalité tient pour tout entier $N \geq 6$. Donc $c = 4$ et $N_0 = 5$ conviennent (condition $N > N_0$). Tout couple valable suffit ; seule l'existence compte.

Les constantes (slide 35) : la notation O les ignore, car elle ne décrit que la croissance à long terme ; mais en pratique elles comptent. Distinguer « sont-elles pertinentes ? » (oui) et « O s'en soucie-t-elle ? » (non).

Le problème des pièces (slides 32 et 33)

N pièces, une fausse plus légère, une balance à deux plateaux. Force brute : $N - 1$ pesées, $O(N)$. La slide donne comme solution $O(\log_2 N)$, et mieux $O(\log_3 N)$.

Méthode par tiers (méthode générale)

Répartir les pièces en trois groupes aussi égaux que possible et peser deux groupes de même taille. Le plateau le plus léger contient la fausse pièce ; en cas d'équilibre, elle est dans le troisième groupe. Chaque pesée divise les candidats par 3 : il faut $\lceil \log_3 N \rceil$ pesées. Avec 3 pièces (indice de la slide), une seule pesée suffit : pièce 1 contre pièce 2.

N	force brute $N-1$	moitiés $\lceil \log_2 N \rceil$	tiers $\lceil \log_3 N \rceil$
9	8	4	2
27	26	5	3
100	99	7	5
1000	999	10	7

Piège

$O(\log_2 N)$ et $O(\log_3 N)$ sont la même classe : $\log_2 N = \log_2 3 \times \log_3 N$ avec $\log_2 3 \approx 1{,}585$, un facteur constant. Le nombre réel de pesées diffère, la classe O non. C'est exactement la remarque de la slide 35 sur les constantes.

Algorithmes de tri (slide 34)

	meilleur	moyen	pire	mémoire (pire)
Quicksort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(N \log N)$
Mergesort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Tri à bulles	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$

$O(1)$ en mémoire : tri en place, aucune mémoire supplémentaire proportionnelle à N.

Écart concret pour $N = 1000$

$N^2 = 10^6$ contre $N \log_2 N \approx 9\,966$: environ 100 fois plus d'opérations. Pour $N = 10^6$, le rapport $N / \log_2 N$ dépasse 50 000. Sur la courbe de la slide 33, $\log_2 N$ reste sous 7 pour $N \leq 100$ alors que N monte à 100.

Une passe de tri à bulles sur l'exemple de la slide 34

Liste [3,9; −2; 1; 7; 12; 5; −2,37]. On compare les voisins de gauche à droite et on échange s'ils sont mal rangés. Passe 1 : quatre échanges, résultat [−2; 1; 3,9; 7; 5; −2,37; 12] : le maximum 12 est en place. Le −2,37 ne recule que d'une case par passe : il faut 6 passes avec échanges, puis une 7e sans échange pour conclure.

Piège

La mémoire d'un quicksort : la slide indique $O(N \log N)$. Les références usuelles donnent $O(\log N)$ en moyenne et $O(N)$ au pire selon l'implémentation. **À VÉRIFIER** ; à l'examen, répondre avec la valeur de la slide. Pour l'embarqué, retenir le compromis : le tri à bulles est lent mais ne demande aucune mémoire en plus.

7. Apprentissage automatique : les principes

Source : `02_Challenges`, p. 4 à 8 du PDF

L'IA aujourd'hui (cloud) est	Un système embarqué a
gourmande en calcul	un processeur moins performant
gourmande en mémoire	une mémoire limitée
gourmande en données (entraînement)	une énergie limitée (batterie)
gourmande en énergie	des contraintes temps réel, un temps de réaction parfois critique
lente pour beaucoup de problèmes temps réel	une bande passante limitée et une communication peu fiable

À retenir

- **Supervisé** : pour chaque entrée d'entraînement, la sortie désirée est connue (exemple = valeurs d'entrée + sortie désirée).
- **Par renforcement** : une suite d'étapes dont on ne connaît pas la sortie désirée à chaque pas ; on reçoit parfois un retour (récompense ou regret), pas à chaque étape.
- **Non supervisé** : aucune information sur la sortie désirée, seulement les entrées ; on cherche par exemple des groupes (clusters) d'exemples semblables entre eux et dissemblables d'un groupe à l'autre.

Entraînement supervisé (p. 6). Jeu d'exemples $\{(\mathbf{x}_n, d_n)\}$, $n = 1, \ldots, N$; par exemple $\mathbf{x}_n = [1{,}5;\ 1;\ \text{NICE}; \ldots]$ et $d_n = \text{TRUE}$. Les attributs peuvent être numériques ou symboliques. Un algorithme d'entraînement, *non spécifique au problème*, construit une **hypothèse** $h(\mathbf{x})$: la règle générale tirée des exemples. **Utilisation (p. 7)** : pour une situation nouvelle $\mathbf{x}_{\text{new}} = [2{,}7;\ 0;\ \text{UGLY}; \ldots]$, le système propose $h(\mathbf{x}_{\text{new}})$, par exemple NO. Le système *généralise* à des cas jamais vus.

Système entraînable (p. 8)	Apprentissage typique
Réseaux de neurones	les deux (supervisé plus courant)
Réseaux bayésiens, machines à noyau	(non précisé)
Arbres de décision	supervisé
Plus proche voisin	supervisé
K-means	non supervisé
Clustering hiérarchique	non supervisé

Piège

Clustering = non supervisé (pas d'étiquette) ; classification = supervisé. Un réseau de neurones peut être l'un ou l'autre. Les régressions linéaire et logistique du TP 2 sont supervisées : chaque x a son y.

8. Défis 1 à 5 : entraînement, taille, énergie, vitesse, encombrement

Source : `02_Challenges`, p. 9 à 17 du PDF

Les huit défis (p. 9) : temps d'entraînement, taille du modèle, efficacité énergétique, vitesse d'inférence, taille et poids physiques, sûreté, confiance, gouvernance.

Défi 1, temps d'entraînement (p. 10 et 11)

Réseau (classification d'images)	erreur	entraînement
ResNet18	10,76 %	2,5 jours
ResNet50	7,02 %	5 jours
ResNet101	6,21 %	1 semaine
ResNet152	6,16 %	1,5 semaine

Mesuré sur quatre GPU M40. De ResNet18 à ResNet152 : temps multiplié par 10,5/2,5 = 4,2 pour 4,6 points d'erreur en moins ; de ResNet101 à ResNet152, 3,5 jours de plus pour 0,05 point. **Conclusion : on n'entraîne généralement pas sur l'embarqué.** Parfois on poursuit l'entraînement d'un

modèle déjà entraîné avec de nouvelles données pour l'adapter (ce n'est pas un entraînement depuis zéro).

Défi 2, taille du modèle (p. 12 à 14)

Tendance (p. 12, « montre seulement la tendance ») : en image, AlexNet (2012, 8 couches, 1,4 GFLOP, environ 16 % d'erreur) puis ResNet (2015, 152 couches, 22,6 GFLOP, environ 3,5 %) : taille du modèle ≈ ×20. En parole, Deep Speech 1 (2014, 80 GFLOP, 7000 h de données, environ 8 %) puis Deep Speech 2 (2015, 465 GFLOP, 12 000 h, environ 5 %) : opérations d'entraînement ≈ ×6 (465/80 = 5,8). FLOP : opérations en virgule flottante.

Modèle (p. 13)	prof.	param. (M)	GFLOPS	Mo	top-5 err.
AlexNet	7	58,3	0,725	217	19,2 %
VGG16	16	134,2	15,5	512,2	9,9 %
ResNet18	18	11,7	1,814	23	10,92 %
GoogleNet	22	6	1,57	40	12,9 %
ResNet34	34	21,8	3,664	35	8,58 %
ResNetV1 50	50	25,6	3,858	97,7	7,7 %
ResNetV1 101	101	45	7,57	155	7 %
ResNetV1 152	152	60	11,3	230	6,7 %

Précision top-5 égale 100 moins l'erreur top-5 (la slide donne les deux ; ResNet18 et ResNet34 font 100,02 et 99,98 à cause des arrondis). Plus profond ne veut pas dire plus gros : GoogleNet (22 couches) a 6 M de paramètres, AlexNet (7 couches) en a 58,3 M. Les gros modèles sont durs à distribuer par mise à jour à distance (OTA, p. 14 : un téléphone refuse de télécharger plus de 100 Mo hors Wi-Fi).

Défi 3, efficacité énergétique (p. 15)

AlphaGo : 1920 CPU et 280 GPU, 3000 $ d'électricité par partie. Sur un appareil embarqué, la consommation vide la batterie ; en centre de données, elle augmente le **TCO** (*total cost of ownership* : tous les coûts directs et indirects d'achat et d'exploitation sur la durée de vie). Suite au concept 11.

Défi 4, vitesse d'inférence (p. 16)

Besoins : traitement continu (vidéosurveillance), décision rapide (voiture autonome), faible latence (procédés industriels rapides). Problème : les gros modèles sont parfois trop lents et passent mal à l'échelle quand le problème se complique.

Défi 5, taille et poids physiques (p. 17)

Le calcul haute performance vit dans de grandes salles climatisées ; l'embarqué doit être petit et tenir dans des environnements hostiles. AlphaGo (2016) : environ 16 000 kg de matériel, 33 m^3 de baies, environ 1 MW (estimations rapportées par la slide). Cerveau humain : 1,3 kg, 1,3 L, 20 W ; corps : 60 à 80 kg, environ 100 W.

Calcul de la slide 17

1 MW/20 W $= 10^6/20 =$ 50 000 cerveaux humains. Par rapport à la consommation d'un corps entier (100 W) : $10^6/100 =$ 10 000 humains.

9. Défis 6 à 8 : sûreté, confiance, gouvernance

Source : `02_Challenges`, p. 18 à 23 du PDF

Défi 6, sûreté (p. 18)

- **Limites matérielles** : risque de panne ou d'erreur dû au calcul limité.
- **Décisions temps réel** : un temps de traitement insuffisant mène à des décisions hâtives ou fausses, surtout en conduite autonome.
- **Interférence et altération** : l'accès physique à l'appareil permet une intervention malveillante.
- **Sécurité « raisonnable »** : on ne pense pas aux menaces futures.

Défi 7, confiance (p. 19 à 22)

Risque de trop ou de trop peu se fier au système, aggravé en embarqué par l'absence d'interface riche : on ne peut pas lui poser de questions comme à un chatbot.

Notion	Définition	Atténuation
Biais	erreur systématique dans la distribution des données ou le fonctionnement du modèle, qui peut léser certains groupes	identifier les groupes concernés, analyser l'impact sur eux
Responsabilité	déterminer qui répond d'un dommage ou d'une décision erronée	cadre légal contre le « trou de responsabilité » : personne ne sait qui répond du dommage
Transparence	pouvoir comprendre et retracer les résultats et la logique du système	outils d'explication, difficiles car ils exigent aussi un effort de l'utilisateur

Autres points (p. 19) : modèles « boîtes noires », vie privée (quelles données sont collectées, stockées, transmises), cohérence du comportement dans tous les scénarios, biais et équité.

Scandale des allocations familiales aux Pays-Bas (p. 21 et 22)

Système algorithmique introduit en 2013 par l'administration fiscale pour détecter les demandes incorrectes et la fraude. La nationalité non néerlandaise était un facteur de risque : **biais**. L'agent chargé de la revue manuelle ne savait pas pourquoi le score était élevé : **transparence**. Les agents ne se sentaient pas responsables de la décision, attribuée au modèle : **responsabilité**, et **dépendance excessive** (over-reliance). Le gouvernement a démissionné le 15 janvier 2021.

Défi 8, gouvernance (p. 23)

Défis : manque de réglementation standardisée pour beaucoup d'applications ; mises à jour et correctifs sans perturbation ; respect de l'éthique, en particulier en surveillance et reconnaissance faciale. Atténuations : cadre légal pour le trou de responsabilité, tests rigoureux en conditions réelles avant déploiement, contrôle humain (difficile en embarqué), dialogue avec utilisateurs, régulateurs et public, cadres éthiques.

Piège

Sûreté (safety) et confiance (trust) se recoupent. Classer selon le mot-clé de la slide : altération physique → sûreté ; boîte noire, biais, vie privée → confiance ; réglementation, mises à jour → gouvernance.

10. Où calculer : cloud ou local

Source : `02_Challenges`, p. 24 à 27 du PDF

Mode cloud (p. 24). Des SoC (*System on a Chip*) avec capteurs envoient les données à un centre de calcul qui héberge l'application et l'algorithme d'IA (GPU, TPU). **Mode local (p. 26 et 27).** Dans l'appareil : capteurs, SoC, puce d'accélération IA, algorithme d'IA *allégé*, application.

	Négatif	Positif
Cloud	n'aide pas l'énergie locale limitée ; aggrave le temps réel et le temps de réaction ; bande passante limitée, communication peu fiable	calcul plus puissant ; résout les limites de mémoire
Local	n'aide pas l'énergie locale limitée ; calcul faible ; n'aide pas la mémoire ; compromis entre précision et complexité	meilleur temps de réaction ; la bande passante pose moins de problème

Exemple du mode cloud : Siri (p. 25, d'après Bin Li)

Sur le téléphone : Core Audio fournit le son à un **détecteur de déclenchement vocal** local ; le Voice Trigger Framework fournit l'audio, règle le seuil de détection et envoie les événements de réveil au client Siri. Dans le cloud : le serveur Siri vérifie les premiers mots, peut annuler le déclenchement et journalise. Le serveur peut envoyer une nouvelle configuration du modèle (datapack).

Piège

Aucun des deux modes ne résout la limite d'énergie locale : les deux

listes « négatif » la contiennent. Question classique : « quel mode améliore le temps de réaction ? » (local) ; « lequel aide la mémoire ? » (cloud).

11. Où part l'énergie

Source : `02_Challenges`, p. 28 et 33 du PDF

« Modèle plus gros ⇒ plus d'accès mémoire ⇒ plus d'énergie. »

Opération (Horowitz, 45 nm)	énergie (pJ)	aire (μm^2)
addition 8 bits	0,03	36
addition 16 bits	0,05	67
addition 32 bits	0,1	137
addition flottante 16 bits	0,4	1360
addition flottante 32 bits	0,9	4184
multiplication 8 bits	0,2	282
multiplication 32 bits	3,1	3495
multiplication flottante 16 bits	1,1	1640
multiplication flottante 32 bits	3,7	7700
lecture SRAM 32 bits (8 Ko)	5	
lecture DRAM 32 bits	640	

La slide p. 28 cite aussi le banc de registres 32 bits à 1 pJ et résume : un accès DRAM coûte autant qu'environ 1000 opérations arithmétiques.

Ratios à savoir refaire

DRAM contre addition entière 32 bits : $640/0{,}1 = 6400$. DRAM contre addition flottante 32 bits : $640/0{,}9 \approx 711$. DRAM contre SRAM : $640/5 = 128$. Multiplication flottante 32 bits contre multiplication 8 bits : $3{,}7/0{,}2 = 18{,}5$ en énergie et $7700/282 \approx 27$ en surface.

Budget d'une inférence (méthode générale, chiffres de la slide)

Un modèle fait 10^6 multiplications-additions en FP32 et lit ses 10^6 poids

en DRAM. Calcul : $10^6 \times (3{,}7 + 0{,}9) = 4{,}6 \cdot 10^6$ pJ $= 4{,}6\ \mu$J. Mémoire : $10^6 \times 640 = 640\ \mu$J. Total 644,6 μJ, dont 99,3 % pour la DRAM. Si les poids tiennent en SRAM : $4{,}6 + 5 = 9{,}6\ \mu$J, environ 67 fois moins. En entiers 8 bits, le calcul tombe à $10^6 \times (0{,}2 + 0{,}03) = 0{,}23\ \mu$J, 20 fois moins qu'en FP32.

Puissance de la DRAM (p. 28) : $P = \frac{1}{2} \sum_i C_i V_i^2 f_i$, la fréquence f_i étant entourée. [rappel général] La puissance est proportionnelle à la fréquence des commutations, donc aux accès : diviser f par 2 divise P par 2 ; diviser V par 2 divise P par 4 ; passer de 1,2 V à 1,0 V multiplie P par $(1{,}0/1{,}2)^2 \approx 0{,}69$.

> **À retenir**
>
> Pour économiser l'énergie en embarqué : garder les poids près du calcul (registres, SRAM), réduire leur nombre (élagage) et leur taille (quantification), préférer l'entier au flottant.

12. Matériel, premières applications, capteurs intelligents

Source : `02_Challenges`, p. 29 à 31 du PDF

Taxonomie (p. 29). Matériel généraliste : **CPU** orienté latence (sert un fil d'exécution séquentiel avec une faible latence), **GPU** orienté débit, **TPU**. Matériel spécialisé : **FPGA**, logique programmable ; **ASIC**, logique figée.

[rappel général] Latence : temps pour traiter une donnée ; débit : nombre de données traitées par unité de temps. Un FPGA se reconfigure après fabrication, un ASIC non : il est plus efficace mais son circuit ne change plus.

Premières applications embarquées (p. 30). Détection de mot de réveil (« Ok Google », « Hey Siri ») ; détection d'accident de voiture par fusion de données (micro, capteurs inertiels, GPS) avec alerte des secours ; comptage de pas et détection d'activité (capteurs inertiels et biocapteurs). Ces tâches difficiles ne nous paraissent plus « intelligentes » : ce qui émerveillait hier devient un minimum attendu, comme 5 + 3 pour un enfant de 3 ans puis pour un adulte.

Sensor hubs (p. 31). D'abord des composants séparés : le coprocesseur de mouvement M7 (NXP LPC18A1) à côté de l'Apple A7, aujourd'hui intégré

au SoC des smartphones ; certains restent vendus par les fabricants de capteurs. But commun : des informations utiles pour une très faible consommation. Typiquement un ARM Cortex-M couplé au capteur avec un logiciel préchargé. Va-et-vient dans et hors du SoC : microNPU Ethos U-55 (*Neural Processing Unit*) intégré aux IP ARM.

13. Représentation des nombres et quantification

Source : `02_Challenges`, p. 32 et 33 du PDF ; `01_Intro`, slide 28

Flottant : $(-1)^S \times (1.M) \times 2^E$ (p. 32).

Format	bits (S, E, M)	total	étendue	précision
FP32	1, 8, 23	32	10^{-38} à 10^{38}	0,000006 %
FP16	1, 5, 10	16	$6 \cdot 10^{-5}$ à $6 \cdot 10^{4}$	0,05 %
Int32	1, 31	32	0 à $2 \cdot 10^{9}$	1/2
Int16	1, 15	16	0 à $6 \cdot 10^{4}$ (slide)	1/2
Int8	1, 7	8	0 à 127	1/2
Virgule fixe	S, I, F			

[rappel général] L'exposant est stocké avec un décalage (127 en FP32, 15 en FP16). Les précisions relatives correspondent à la moitié de l'écart entre deux flottants voisins : $2^{-24} \approx 6 \cdot 10^{-8}$, soit 0,000006 %, en FP32 ; $2^{-11} \approx 4{,}9 \cdot 10^{-4}$, soit 0,05 %, en FP16. Les valeurs extrêmes calculées avec NumPy : FP16 va de $6{,}1 \cdot 10^{-5}$ (plus petit normalisé) à 65 504 ; FP32 de $1{,}18 \cdot 10^{-38}$ à $3{,}4 \cdot 10^{38}$. Pour les entiers, « 1/2 » s'interprète comme l'erreur d'arrondi maximale d'une demi-unité (interprétation, **À VÉRIFIER**).

> **Piège**
>
> Int8 : 1 bit de signe et 7 bits de magnitude, donc 127 au plus ($2^7 - 1$). Int16 : la slide indique $6 \cdot 10^4$, mais 15 bits de magnitude donnent $2^{15} - 1 = 32\,767$; $6 \cdot 10^4$ correspond à 16 bits sans signe (65 535). Int32 : $2^{31} - 1 \approx 2{,}1 \cdot 10^9$, cohérent avec la slide. Donner la valeur de la slide si on la demande, en sachant la calculer.

Taille mémoire d'un modèle

Méthode générale : taille = nombre de paramètres × octets par paramètre (4 en FP32, 2 en FP16, 1 en int8).

> **VGG16, 134,2 M de paramètres**
>
> FP32 : $134{,}2 \cdot 10^6 \times 4 = 536{,}8 \cdot 10^6$ octets = 511,9 Mio : c'est la valeur 512,2 de la slide p. 13 (en mébioctets). FP16 : 268,4 Mo. Int8 : 134,2 Mo, quatre fois moins qu'en FP32. C'est l'intérêt de la **quantification** annoncée en `01_Intro`, slide 28.

Les autres lignes du tableau ne suivent pas toutes cette règle (ResNet18 : $11{,}7 \times 4 = 46{,}8$ Mo en FP32 contre 23 dans la slide ; GoogleNet : 24 Mo contre 40) : format de stockage non précisé, **À VÉRIFIER**.

Énergie par format (p. 33) : à opération égale, le flottant 16 bits coûte moins que le 32 bits (multiplication : 1,1 contre 3,7 pJ, 1640 contre 7700 μm^2), et l'entier 8 bits encore moins (0,2 pJ, 282 μm^2).

14. TP 1 : Python

Source : `lab01-exercises.ipynb` et `lab01-solutions.ipynb`, cellules 0 à 71 ; notebook de Mathieu Blondel

Le TP présente les notebooks (Jupyter, Google Colab), puis les bases de Python. Une cellule de code s'exécute avec Ctrl ou Cmd + Entrée ; les variables d'une cellule servent aux suivantes, donc l'ordre d'exécution compte (« Run all » pour tout rejouer dans l'ordre, cellule 10).

> **À retenir**
>
> - Opérateurs : `+`, `*`, `/` (division réelle), `**` (puissance), `//` (division entière).
> - Listes : ordonnées, taille variable, types mélangés, indices depuis 0 ; tranche `l[i:j]` de `i` inclus à `j` exclu ; `in`, `len`, `append`.
> - Chaînes : indexées et découpées comme des listes ; concaténation par `+`.
> - `if`, `elif`, `else` ; `==` teste l'égalité, `=` affecte ; les blocs sont délimités par l'indentation.

- Boucles `while` et `for` ; fonctions avec `def` et `return`, arguments nommables (`multiply(a=3, b=2)`).

```
my_list = [1, 2, 3, "apple"]
my_list.append(4)
print(my_list[2], my_list[1:3], my_list[-1], len(my_list))
s = "some text"
print(s[3], s[5:], s + "!")
print(7 / 2, 7 // 2, 2 ** 10, 7 % 3)
print(5 in my_list, "apple" in my_list)
```

Sortie :

```
3 [2, 3] 4 5
e text some text!
3.5 3 1024 1
False True
```

Exercices du TP et solutions

ReLU : relu$(x) = x$ si $x \geq 0$, 0 sinon. Norme euclidienne par boucle : $\sqrt{\sum x_i^2}$; attendu 5,729746940310715 pour $[0{,}5;\ -1{,}2;\ 3{,}3;\ 4{,}5]$. Maximum par boucle. Bonus : tri à bulles, laissé vide dans les deux notebooks ; solution ci-dessous, avec arrêt anticipé qui donne le meilleur cas $O(N)$ du concept 6.

```
import numpy as np

def relu(x):
    if x >= 0:
        return x
    return 0

def euclidean_norm(vector):
    total = 0
    for x in vector:
        total += x * x
    return np.sqrt(total)

def bubble_sort(lst):
    lst = list(lst)
    n = len(lst)
    for i in range(n - 1):
```

```
        swapped = False
        for j in range(n - 1 - i):
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]
                swapped = True
        if not swapped:
            break
    return lst

print(relu(-3), relu(2.5))
print(euclidean_norm([0.5, -1.2, 3.3, 4.5]))
print(bubble_sort([1, -3, 3, 2]))
```

Sortie :

```
0 2.5
5.729746940310715
[-3, 1, 2, 3]
```

Piège

Les sorties enregistrées dans `lab01-solutions.ipynb` viennent d'exécutions dans un autre ordre : la cellule 22 affiche `[1, 2, 3, 'apple', 4, 4]` (`append` lancé deux fois), la cellule 32 affiche une longueur de 4, et la cellule 102 affiche pour `np.linspace(0, 1, 10)` des valeurs de 1 à 10 qui sont celles de `np.logspace(0, 1, 10)`. Ne pas apprendre ces sorties ; refaire le calcul.

15. TP 1 : NumPy et Matplotlib

Source : `lab01-solutions.ipynb`, cellules 72 à 158

NumPy stocke des tableaux de nombres d'un seul type (`int32`, `int64`, `float32`, `float64`) et calcule dessus en C : le code est plus court et plus rapide qu'une boucle Python (cellules 73, 89, 112).

À retenir

- Création : `np.array`, `np.zeros((2, 3))`, `np.ones`, `np.arange(3, 5)` (3 et 4), `np.linspace(a, b, n)` (n points de a à b inclus, pas $(b-a)/(n-1)$), `reshape`.

- `shape` donne les dimensions ; `A[i, j]` un élément, `A[i]` une ligne, `A[:, j]` une colonne.
- Opérations élément par élément (`+`, `*`, `np.sin` : fonctions universelles) ; `np.dot` ou `@` : produit scalaire, ou produit matriciel entre tableaux 2D ; `.T` : transposée.
- Masque booléen : `x[x >= 5]` ; `axis=0` agrège le long des lignes (résultat par colonne), `axis=1` le long des colonnes (résultat par ligne).

```
import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6]])
print(A.shape, A[1, 2], A[1], A[:, 2])
a, b = np.array([1, 2, 3]), np.array([4, 5, 6])
print(a + b, a * b, np.dot(a, b), a @ b)
x = np.arange(10)
print(x[x >= 5])
print(np.maximum(0, np.array([1, -3, 2.5])))
M = np.array([[0.5, -1.2, 4.5], [-3.2, 1.9, 2.7]])
print(np.sum(M, axis=0), np.sum(M, axis=1))
print(np.sqrt(np.sum(M ** 2, axis=1)))
print(np.linspace(0, 1, 5), np.linspace(0, 10, 51).shape)
```

Sortie :

```
(2, 3) 6 [4 5 6] [3 6]
[5 7 9] [ 4 10 18] 32 32
[5 6 7 8 9]
[1.  0.  2.5]
[-2.7  0.7  7.2] [3.8 1.4]
[4.68401537 4.59782557]
[0.   0.25 0.5  0.75 1.  ] (51,)
```

La ligne `np.maximum` résout l'exercice 2 (ReLU vectorisée) ; la ligne `np.sqrt(np.sum(M ** 2, axis=1))` résout l'exercice 4 (normes par ligne), laissé vide dans le notebook. Exercice 5 : la moyenne des 4 variables d'iris s'obtient par `X.mean(axis=0)` (non exécuté ici, scikit-learn absent de cette machine).

Matplotlib

`plt.figure()`, `plt.plot(x, y, label=...)`, `plt.xlabel`, `plt.ylabel`, `plt.title`, `plt.legend`, `plt.show()` ; `plt.scatter` pour un nuage de

points, coloré par classe (cellules 145 et 147) ; `plt.imshow` pour une image (un chiffre de `digits` est un tableau 8×8, cellule 156). Constat du notebook sur iris : avec les deux premières variables, la classe 0 est séparable linéairement des deux autres (cellule 148).

[rappel général] Softplus : $\ln(1 + e^x)$, version lisse de ReLU, strictement positive et dérivable partout ; $\text{softplus}(0) = \ln 2 \approx 0{,}693$ alors que $\text{relu}(0) = 0$.

16. TP 2 : régression linéaire, solution analytique

Source : `lab02-solutions.ipynb`, cellules 0 à 4

Données (exercice 1). 51 entrées régulières de 0 à 10 (`np.linspace(0, 10, 51)`, pas 0,2), étiquettes $y = 0{,}7x + 2$ plus un bruit gaussien (`np.random.randn`). Entrées de test : `np.linspace(0, 10, 101)`.

> **À retenir**
>
> Modèle $y \approx \mathbf{w}^\top \phi(x)$ avec $\phi(x) = [1,\ x]$. On empile les $\phi(x_i)$ en lignes dans $\mathbf{X}$ (une colonne de 1, puis les x_i). Solution des moindres carrés :
>
> $$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$
>
> Prédiction : $\mathbf{y}_{\text{test}} = \mathbf{X}_{\text{test}} \mathbf{w}$.

[rappel général] D'où vient la formule : on minimise $E(\mathbf{w}) = \sum_i (y_i - \mathbf{w}^\top \phi(x_i))^2 = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$. Le gradient $-2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$ s'annule quand $\mathbf{X}^\top \mathbf{X}\,\mathbf{w} = \mathbf{X}^\top \mathbf{y}$ (équations normales). La colonne de 1 porte l'ordonnée à l'origine.

À la main : $x = (0, 1, 2, 3)$, $y = (1, 2, 2, 4)$

$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} N & \sum x \\ \sum x & \sum x^2 \end{pmatrix} = \begin{pmatrix} 4 & 6 \\ 6 & 14 \end{pmatrix}$, $\mathbf{X}^\top \mathbf{y} = \begin{pmatrix} \sum y \\ \sum xy \end{pmatrix} = \begin{pmatrix} 9 \\ 18 \end{pmatrix}$. Déterminant $4 \times 14 - 6 \times 6 = 20$. Inverse d'une 2×2 : $\frac{1}{20} \begin{pmatrix} 14 & -6 \\ -6 & 4 \end{pmatrix}$. Donc $w_0 = (14{\times}9 - 6{\times}18)/20 = 18/20 = 0{,}9$ et $w_1 = (-6{\times}9 + 4{\times}18)/20 = 18/20 = 0{,}9$. Droite $y = 0{,}9 + 0{,}9x$; prédiction en $x = 5$: 5,4.

```
import numpy as np
x = np.array([0.0, 1, 2, 3])
y = np.array([1.0, 2, 2, 4])
X = np.vstack([np.ones(len(x)), x]).T
w = np.linalg.solve(X.T @ X, X.T @ y)
print(X.T @ X)
print(X.T @ y, w)
y_hat = X @ w
rmse = np.sqrt(np.mean((y - y_hat) ** 2))
print(y_hat, round(rmse, 4))
for lam in [0, 1, 10]:
    w = np.linalg.solve(X.T @ X + lam * np.eye(2), X.T @ y)
    print(lam, w.round(4))
```

Sortie :

```
[[ 4.  6.]
 [ 6. 14.]]
[ 9. 18.] [0.9 0.9]
[0.9 1.8 2.7 3.6] 0.4183
0 [0.9 0.9]
1 [0.6923 0.9231]
10 [0.36 0.66]
```

Piège

- `np.vstack([np.ones(N), x]).T` : sans le `.T`, on obtient une matrice $2 \times N$ au lieu de $N \times 2$.
- Le notebook conseille `np.linalg.solve` plutôt que l'inverse explicite : plus stable et moins coûteux. $\mathbf{X}^\top\mathbf{X}$ est symétrique définie positive, d'où la piste de la factorisation de Cholesky (cellule 3).
- La solution met un bruit d'écart type 5 (`randn(N)*5`) là où l'énoncé en met 0,8 : la droite estimée s'écarte alors davantage de la vraie ($w_0 = 2$, $w_1 = 0{,}7$).

17. TP 2 : régression polynomiale et régularisation

Source : `lab02-solutions.ipynb`, cellules 5 à 12

Non linéaire (exercices 3 et 4). Données $y = (x-9)(x-6)(x-1)$ plus un bruit d'écart type 5. On garde la même méthode et on change seulement la transformation : $\phi(x) = [1,\ x,\ x^2,\ x^3]$. La matrice $\mathbf{\Phi}$ a une ligne par exemple et $D+1 = 4$ colonnes ; `np.vander(x, D+1, increasing=True)` la construit. Le modèle reste *linéaire en* $\mathbf{w}$, c'est pourquoi la même formule marche.

Lignes de Φ et polynôme cible

$x = 0{,}2$ donne $[1;\ 0{,}2;\ 0{,}04;\ 0{,}008]$ (affichage du notebook), $x = 2$ donne $[1;\ 2;\ 4;\ 8]$. En développant, $(x-9)(x-6)(x-1) = x^3 - 16x^2 + 69x - 54$: les poids idéaux sont $\mathbf{w} = (-54;\ 69;\ -16;\ 1)$, et $y(2) = (-7)(-4)(1) = 28$.

Régularisation (exercice 5). L'estimation au maximum a posteriori (MAP) donne

$$\mathbf{w} = (\mathbf{\Phi}^\top \mathbf{\Phi} + \lambda \mathbf{I})^{-1} \mathbf{\Phi}^\top \mathbf{y},$$

testée avec $\lambda = 0, 1, 10, 100$ (`np.eye(D+1)` pour $\mathbf{I}$).

[rappel général] Cette formule minimise $\|\mathbf{y} - \mathbf{\Phi}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|^2$ (régression ridge) : le terme en λ pénalise les grands poids. Plus λ est grand, plus les poids rétrécissent vers 0 et plus la courbe s'aplatit : on réduit le surapprentissage, mais un λ trop grand fait sous-apprendre. $\lambda = 0$ redonne les moindres carrés.

Mesuré sur une exécution du code de la solution, graine NumPy fixée

Données du TP (bruit 5), ajustement cubique. Norme de $\mathbf{w}$ et RMSE sur l'entraînement : $\lambda = 0$: 91,8 et 4,98 ; $\lambda = 1$: 64,3 et 7,69 ; $\lambda = 10$: 20,6 et 16,5 ; $\lambda = 100$: 4,1 et 21,4. Les poids fondent, l'erreur d'entraînement monte : ici le modèle cubique est déjà adapté, la régularisation le dégrade. Les valeurs changent avec le tirage.

Ridge à la main, données du concept 16, $\lambda = 1$

$\mathbf{X}^\top \mathbf{X} + \mathbf{I} = \begin{pmatrix} 5 & 6 \\ 6 & 15 \end{pmatrix}$, déterminant $75 - 36 = 39$. $w_0 = (15 \times 9 - 6 \times 18)/39 = 27/39 \approx 0{,}692$; $w_1 = (-6 \times 9 + 5 \times 18)/39 = 36/39 \approx 0{,}923$. Avec $\lambda = 10$: $(0{,}36;\ 0{,}66)$.

Piège

Avec `np.eye(D+1)`, le notebook régularise aussi le biais w_0 : dans l'exemple, l'ordonnée à l'origine passe de 0,9 à 0,69 alors que la pente bouge à peine. Beaucoup de bibliothèques ne pénalisent pas le biais. La question de la cellule 11 (« comment λ agit-il ? ») n'a pas de réponse dans la solution : c'est une question ouverte probable.

18. TP 2 : jeu de données réel et RMSE

Source : `lab02-solutions.ipynb`, cellules 13 à 17

Exercice 6. Jeu `diabetes` de scikit-learn (`load_diabetes`) ; découpage aléatoire 80 % entraînement, 20 % test avec `train_test_split(x, y, test_size=0.2, random_state=38)`. [rappel général] Ce jeu compte 442 patients et 10 variables (documentation scikit-learn ; le notebook n'affiche pas la forme).

Deux modèles : la formule du concept 16, avec une colonne de 1 ajoutée devant les 10 variables (11 paramètres), et `LinearRegression().fit(x_train, y_train)` puis `.predict(x_test)`. Comparaison par la **RMSE** :

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}.$$

Sorties enregistrées dans le notebook : 56,94023984486523 à la main et 56,940239844865225 avec scikit-learn ; la version complétée (`random_state=28`) donne 57,76. Mêmes résultats aux arrondis près : `LinearRegression` résout le même problème des moindres carrés et ajoute elle-même l'ordonnée à l'origine.

RMSE à la main

$y = (3,5,2,7)$, $\hat{y} = (2{,}5;\ 5;\ 3;\ 6)$. Erreurs 0,5; 0; −1; 1 ; carrés 0,25; 0; 1; 1 ; moyenne (MSE) $2{,}25/4 = 0{,}5625$; RMSE $\sqrt{0{,}5625} = 0{,}75$. Sur l'exemple du concept 16 : résidus 0,1; 0,2; −0,7; 0,4, somme des carrés 0,70, MSE 0,175, RMSE 0,418.

Piège

La RMSE s'exprime dans l'unité de y (ici l'indicateur de progression du diabète) ; la MSE dans son carré. On l'évalue sur le jeu de *test*, jamais sur celui qui a servi à calculer $\mathbf{w}$. Le `random_state` fixe le découpage : deux graines donnent deux RMSE différentes (56,94 contre 57,76).

19. TP 2 : régression logistique

Source : `lab02-solutions.ipynb`, cellules 18 à 26

Classification binaire avec un modèle linéaire. Données : deux nuages gaussiens de 100 points, centrés en $(-1,-2)$ (classe 0) et $(0,0)$ (classe 1) ; test sur une grille 100×100 de $[-5,5]^2$ pour dessiner la frontière (`np.meshgrid`, `plt.contourf`).

À retenir

- Transformation $\phi(\mathbf{x}) = [1,\ \mathbf{x}]$; modèle $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \phi(\mathbf{x}))$ avec $\sigma(a) = 1/(1+e^{-a})$.
- Descente de gradient : $\mathbf{w} \leftarrow \mathbf{w} - \eta\, \nabla_{\mathbf{w}} l$, η le pas d'apprentissage, $\mathbf{w}$ initialisé à 0, nombre d'itérations fixé.
- Gradient codé dans la solution : `-Phi.T @ (y*(1-s) - (1-y)*s)` avec `s = sigmoid(Phi @ w)`, qui se simplifie en $\mathbf{\Phi}^\top(\sigma(\mathbf{\Phi w}) - \mathbf{y})$.
- Classe prédite : 1 si $\sigma > 0{,}5$, c'est-à-dire si $\mathbf{w}^\top \phi > 0$: la frontière est la droite $\mathbf{w}^\top \phi(\mathbf{x}) = 0$.

[rappel général] La perte est la log-vraisemblance négative $l(\mathbf{w}) = -\sum_i [\, y_i \ln \sigma_i + (1 - y_i) \ln(1 - \sigma_i)\,]$. Avec $\sigma'(a) = \sigma(a)(1 - \sigma(a))$, son gradient vaut $\sum_i (\sigma_i - y_i)\phi_i$. Pas de formule fermée : d'où l'algorithme itératif. En $\mathbf{w} = 0$, tous les σ_i valent 0,5 et $l = N \ln 2$.

a	-2	-1	0	1	2	3	4,6
$\sigma(a)$	0,1192	0,2689	0,5	0,7311	0,8808	0,9526	0,9900

Un pas de descente à la main

Points $(0,0)$, $(1,0)$ de classe 0 et $(0,2)$, $(2,2)$ de classe 1 ; $\mathbf{\Phi}$ a pour lignes $[1,0,0]$, $[1,1,0]$, $[1,0,2]$, $[1,2,2]$. En $\mathbf{w}=0$: $\sigma=0{,}5$ partout, donc $\sigma-y=(0{,}5;\ 0{,}5;\ -0{,}5;\ -0{,}5)$. Gradient $\mathbf{\Phi}^{\top}(\sigma-y)$: biais $0{,}5+0{,}5-0{,}5-0{,}5=0$; x_1 : $0{,}5\times1-0{,}5\times2=-0{,}5$; x_2 : $-0{,}5\times2-0{,}5\times2=-2$. Avec $\eta=0{,}1$: $\mathbf{w}=(0;\ 0{,}05;\ 0{,}2)$. La perte passe de $4\ln 2\approx 2{,}773$ à $2{,}399$.

```
import numpy as np

def sigmoid(a):
    u = np.exp(-np.abs(a))
    return np.where(a > 0, 1 / (1 + u), u / (1 + u))

Phi = np.array([[1.0, 0, 0], [1, 1, 0], [1, 0, 2], [1, 2, 2]])
y = np.array([0.0, 0, 1, 1])
w = np.zeros(3)
eta = 0.1
for it in range(3):
    grad = Phi.T @ (sigmoid(Phi @ w) - y)
    w = w - eta * grad
    print(it, grad.round(4), w.round(4))
pred = np.where(sigmoid(Phi @ w) > 0.5, 1, 0)
print(pred, np.sum(pred != y))
```

Sortie :

```
0 [ 0.  -0.5 -2. ] [0.   0.05 0.2 ]
1 [ 0.2336 -0.2426 -1.5577] [-0.0234  0.0743  0.3558]
2 [ 0.3702 -0.0917 -1.2733] [-0.0604  0.0834  0.4831]
[0 1 1 1] 1
```

Après trois pas, le point $(1,0)$ est encore mal classé : il faut plus d'itérations.

Piège

- **Sigmoïde stable** (cellule 22) : e^{-a} déborde pour a très négatif (e^{800} vaut `inf` en float64, avec un avertissement). La version du TP ne calcule que $u=e^{-|a|}\le 1$ et renvoie $1/(1+u)$ si $a>0$, $u/(1+u)$ sinon.
- **Comptage d'erreurs** : la solution écrit `num_errs = np.sum(pred-y)`. Les erreurs $+1$ (faux positifs) et -1 (faux négatifs) se compensent : pour `pred` $=(1,0,1,0)$ et `y` $=(0,1,1,0)$, on obtient 0 alors qu'il y a

2 erreurs. Écrire `np.sum(pred != y)`. Sur une exécution de la solution (graine fixée), on a mesuré 28 erreurs sur 200 (86 % de bonnes réponses) alors que `np.sum(pred-y)` vaut -20.

- Le gradient est une *somme* sur les 200 points, pas une moyenne : le pas $\eta = 0{,}1$ agit donc 200 fois plus fort qu'avec une moyenne.

Exercice facultatif (cellule 26) : dériver le gradient de la perte régularisée et entraîner un modèle logistique non linéaire régularisé pour provoquer sous- et surapprentissage. [rappel général] Avec la pénalité $\frac{\lambda}{2}\|\mathbf{w}\|^2$, le gradient devient $\mathbf{\Phi}^\top(\sigma - \mathbf{y}) + \lambda\mathbf{w}$.

20. Classification : le problème

Source : `03_Classification`, p. 2 à 4

Classer, c'est prédire une sortie **catégorielle** parmi des classes fixées d'avance ; la régression prédit une sortie continue (p. 4). Exemples de la slide : panne ou état normal en maintenance prédictive, couleur d'un feu, sain ou malade, de valeur ou sans valeur, mention à un examen. On transforme parfois le problème pour obtenir aussi des probabilités.

Un problème à plusieurs classes (voiture, arbre, piéton, cycliste) se ramène à une suite de décisions binaires : « est-ce une voiture ? », sinon « est-ce un arbre ? », et ainsi de suite. La slide 3 ajoute qu'une sortie « je ne sais pas » est utile en bout de chaîne. Le cours traite surtout le cas binaire.

21. Plus proches voisins (kNN)

Source : `03_Classification`, p. 5 et 6 ; `NN2a`, slide 17

> **À retenir**
>
> Pour classer un nouveau point : (1) trouver les K points d'entraînement les plus proches ; (2) vote majoritaire. Si 4 des 5 voisins sont rouges, le point est rouge « avec une probabilité de 80 % » (p. 5). Pour N exemples de dimension d, le calcul des distances domine : $O(dN)$ par nouveau point (p. 6).

Il n'y a pas de phase d'entraînement coûteuse : kNN est un apprentissage « à mémoire » (memory-based learning) qui stocke les paires entrée, classe et définit un voisinage local du nouveau point. Avec K voisins plutôt qu'un seul, la règle est plus robuste aux valeurs aberrantes (`NN2a`, slide 17). Tout le coût est payé à chaque classification.

Embarqué (p. 6). Contre le manque de ressources, on parallélise (beaucoup de systèmes embarqués sont distribués). Le jeu d'entraînement est découpé en M sous-ensembles placés sur M nœuds ; chaque nœud calcule ses K plus proches en $O(dN_m)$; on garde les K meilleurs parmi les MK candidats, puis on vote.

Compter les opérations, type quiz 03.1

[rappel général] Distance euclidienne au carré entre x_{new} et un exemple : $\sum_{i=1}^{d}(x_i - x_{\text{new},i})^2$, soit d soustractions, d multiplications et $d-1$ additions. La racine est inutile pour comparer (fonction croissante). Pour N exemples : Nd multiplications. Avec $N = 2000$ et $d = 10$: 20 000 multiplications. Sélectionner les K plus petites distances ne demande que des comparaisons. Distance de Manhattan : aucune multiplication.

Piège

L'énoncé exact du quiz 03.1 n'est pas dans le dépôt (slides en version NoQuiz). Lire la question mot à mot : multiplications seules ou toutes les opérations ? distance au carré ou avec racine ? un seul nouveau point ou plusieurs ?

22. Arbres de décision

Source : `03_Classification`, p. 7 à 16

À retenir

- **Arbre de décision** : fonction logique représentée par un arbre. Entrée : un objet ou une situation décrits par des attributs. Sortie : oui ou non, une décision, ou une catégorie (p. 7).
- **Nœud interne** : test d'un attribut. **Arête** : une valeur possible du test. **Feuille** : la valeur ou la catégorie rendue (p. 7).
- **Expressivité** : toute fonction de logique propositionnelle s'écrit comme un arbre (une ligne de la table de vérité par chemin), mais la taille peut être exponentielle en le nombre d'attributs (p. 11).

Exemple des slides 7 à 10. Maintenance d'un appareil selon trois attributs : il claque ou non (rattling), température, âge. Sorties « Shut down », « Urgent Maintenance », « Maintenance », « No problem ». Les règles SI, SINON SI de la slide sont l'arbre écrit en logique. L'entrée $X_{12} = [\text{NOT RATTLING}, 48\,°\text{C}, 9\text{ ans}]$ parcourt un seul chemin jusqu'à une feuille. Il existe aussi des arbres de régression, hors programme (p. 7).

Vocabulaire (p. 11). Exemple : valeurs des attributs et valeur du prédicat cible. Exemple positif ou négatif : cible vraie ou fausse. Ensemble d'entraînement : tous les exemples.

Arbre trivial (p. 12). Un chemin par exemple : l'arbre mémorise les exemples, n'extrait aucun motif et ne classe pas un cas nouveau. Un « vrai » arbre résume beaucoup de cas sous une forme compacte, donc généralise.

Construction (p. 13 à 15). Trouver le plus petit arbre est en général intraitable. On procède de façon **gloutonne** : tester d'abord l'attribut « le plus important », celui qui sépare le mieux positifs et négatifs ; recommencer sur chaque branche ; une branche dont les exemples sont tous positifs (ou tous négatifs) est terminée. Optimum local à chaque nœud : le résultat peut être globalement sous-optimal. Un attribut parfait sépare d'un coup positifs et négatifs, mais il existe rarement.

Exemple de la p. 16. 12 exemples, 6 positifs et 6 négatifs. Le test A_1 donne les branches V_1 (0 positif, 2 négatifs), V_2 (4, 0), V_3 (2, 4) : décider après ce

seul test fait au plus 2 erreurs sur 12. Le test A_2 donne quatre branches à moitié positives : 6 erreurs sur 12, pas mieux qu'avant le test. Un attribut peu informatif laisse dans chaque branche les mêmes proportions que dans l'ensemble de départ. Gains calculés au concept suivant : 0,541 bit pour A_1, 0 pour A_2.

Compter instances et fonctions, type quiz 03.3 et 03.4

[rappel général] Avec n attributs binaires, il y a 2^n instances distinctes (lignes de la table de vérité) et 2^{2^n} fonctions booléennes possibles (chaque ligne reçoit oui ou non). Pour 10 attributs binaires : $2^{10} = 1024$ instances et 2^{1024} fonctions, un nombre de 309 chiffres. Si les attributs ont $k_1, k_2, \ldots$ valeurs, le nombre d'instances est le produit $k_1 k_2 \cdots$. Les énoncés des quiz 03.3 et 03.4 ne sont pas dans le dépôt : vérifier le nombre de valeurs de chaque attribut.

23. Entropie et gain d'information

Source : `03_Classification`, p. 17 à 26 ; calculs vérifiés par `calc_embai_s36.py`

À retenir

Information nécessaire pour décider quand les issues j ont les probabilités $P(j)$ (p. 17) :

$$I(P(1), \ldots, P(k)) = -\sum_{j=1}^{k} P(j) \log_2 P(j) \quad \text{(en bits)}.$$

Ensemble de p positifs et n négatifs (p. 20) : $I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}$.
Un test A à N issues découpe l'ensemble en sous-ensembles (p_k, n_k) (p. 23 et 24) :

$$\text{Reste}(A) = \sum_{k=1}^{N} \frac{p_k + n_k}{p + n} I\left(\frac{p_k}{p_k + n_k}, \frac{n_k}{p_k + n_k}\right),$$

$$\text{Gain}(A) = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - \text{Reste}(A).$$

On teste l'attribut de **gain maximal** (glouton). Le poids $\frac{p_k+n_k}{p+n}$ de chaque branche est indispensable.

Proportions	I (bits)	Proportions	I (bits)
1/2; 1/2	1	1/3; 2/3	0,918
$4 \times 1/4$	2	2/5; 3/5	0,971
1/4; 3/4	0,811	0,2; 0,8	0,722
1; 0	0	0,1; 0,9	0,469

Pièces (p. 18 et 19). Pari de 1000 euros sur pile ou face. Pièce équilibrée : il faut 1 bit. Pièce qui tombe sur face 80 % du temps : $I(0{,}8;\ 0{,}2) = 0{,}722$ bit (quiz 03.5, raté ; voir aussi `fiche-midterm-practice.pdf`). Pièce à deux faces : 0 bit, l'issue est connue. La slide trace $I(p_1, 1-p_1)$: maximum 1 en $p_1 = 0{,}5$, nul en 0 et en 1.

Le piège de la p. 22. Quatre classes de 5000 exemples : il faut 2 bits avant le test. Un test à quatre issues laisse dans chaque branche deux classes de 2500, soit 1 bit par branche. « Gain $= 2 - 4 \times 1 = -2$ bits » est faux : il faut pondérer chaque branche par sa part des exemples, ici $\frac{1}{4}$. Reste $= 4 \times \frac{1}{4} \times 1 = 1$ bit, gain $= 2 - 1 = 1$ bit.

Gains de la p. 16

Avant le test : 6 positifs, 6 négatifs, $I = 1$ bit. A_1 : Reste $= \frac{2}{12} \times 0 + \frac{4}{12} \times 0 + \frac{6}{12} \times I(\frac{1}{3}, \frac{2}{3}) = 0{,}5 \times 0{,}918 = 0{,}459$; gain $= 0{,}541$ bit. A_2 : quatre branches à $\frac{1}{2}, \frac{1}{2}$, reste $= 1$, gain $= 0$. On teste A_1.

Deux remarques (p. 25 et 26).

- $I(1, 0) = 0$: on pose $0 \cdot \log 0 = 0$ (limite, par la règle de L'Hôpital). On peut calculer avec ln au lieu de $\log_2$: ils ne diffèrent que d'un facteur constant, $\log_2 x = \log_2(e) \ln x$ (le classement des attributs ne change pas).
- Un attribut qui prend une valeur différente pour chaque exemple (numéro d'identification) a un gain maximal : chaque branche est pure. Mais l'arbre ne fait que mémoriser et ne généralise pas. Méfiance envers les tests à très nombreuses issues ; certains algorithmes les pénalisent.

24. Problèmes de l'apprentissage supervisé

Source : `03_Classification`, p. 27 à 32

- **Classes déséquilibrées (p. 27).** Même avec beaucoup de données étiquetées : si 6 exemples sur 100 sont de la classe 1, un système qui répond toujours « classe 2 » a une erreur moyenne de 0,06 et ne sert à rien.
- **Bruit (p. 28 à 30).** Données idéales : le système apprend la vraie frontière. Étiquettes fausses (bruit de sortie) ou mesures bruitées de x_1, x_2 : un système « trop malin » apprend la frontière bruitée, c'est le **surapprentissage**.
- **Remède (p. 31).** Beaucoup d'exemples de qualité, et un découpage : une partie sert à l'entraînement, une autre, inconnue du programme, mesure la performance sur des cas nouveaux et dit quand arrêter l'entraînement : c'est la **validation**. On arrête quand l'erreur moyenne de validation, tracée en fonction du nombre d'époques, cesse de baisser.
- **Test final (p. 32).** Une fois l'erreur de validation acceptable, un jeu de test indépendant (ni entraînement, ni validation) mesure la performance.

> **Piège**
>
> Les trois sous-ensembles doivent tous ressembler à la situation réelle (p. 32). Exemple : sonder une population répartie à 20 % en villes, 20 % en bourgs, 60 % en villages, avec un entraînement à 90 % urbain, une validation à 60 % urbaine et 30 % en bourgs, un test à 80 % en villages : mauvais résultat. Chaque sous-ensemble doit reproduire à peu près la répartition 20-20-60.

25. Matrice de confusion et métriques

Source : `03_Classification`, p. 33 à 41 ; précision et F1 de l'exemple calculés par `calc_embai_s36.py`

Réel \ prédit	Positif	Négatif	
Positif	TP	FN (erreur de type II)	Sensibilité $\frac{TP}{TP+FN}$
Négatif	FP (erreur de type I)	TN	Spécificité $\frac{TN}{TN+FP}$
	Précision $\frac{TP}{TP+FP}$	VPN $\frac{TN}{TN+FN}$	Accuracy $\frac{TP+TN}{\text{total}}$

À retenir

- **Accuracy** : justesse globale. **Précision** (valeur prédictive positive) : parmi les prédits positifs, combien le sont vraiment.
- **Sensibilité** (rappel, TPR) : parmi les vrais positifs, combien sont détectés. **Spécificité** (TNR) : parmi les vrais négatifs, combien sont reconnus.
- FPR $= 1 -$ spécificité $= \frac{FP}{FP+TN}$.
- $F_1 = 2\,\frac{\text{précision}\cdot\text{sensibilité}}{\text{précision}+\text{sensibilité}}$, utile quand les classes sont déséquilibrées.
- Entropie croisée pour $y_i \in \{0,1\}$ et N exemples : $-\frac{1}{N}\sum_i [\, y_i \log p(x_i) + (1-y_i)\log(1-p(x_i))\,]$.

Source : p. 39

Exemple du cours (p. 33 à 40). 20 000 patients, une mesure scalaire x, une étiquette sain ou malade. On déclare malade si $x \geq 12$; positif = malade.

Intervalle de x	$[-100, 5)$	$[5, 12)$	$[12, 27)$	$[27, 83)$	$[83, 120)$
sains	3200	2300	2000	1800	700
malades	500	1500	1800	2800	3400

Sains sous le seuil : $3200+2300 = 5500 = TN$; au-dessus : $2000+1800+700 = 4500 = FP$. Malades sous le seuil : $500 + 1500 = 2000 = FN$; au-dessus :

8000 = TP. Sensibilité 8000/10 000 = 0,8 ; spécificité 5500/10 000 = 0,55 ; accuracy 13 500/20 000 = 0,675 (p. 40). Précision 8000/12 500 = 0,64 ; F_1 = $2 \times 0{,}64 \times 0{,}8/1{,}44 = 0{,}711$.

Piège

- Contrôle : $TP+FN+FP+TN$ redonne le total (20 000). Écrire d'abord quelle classe est positive : échanger les rôles échange sensibilité et spécificité.
- Coquilles des slides : la p. 38 étiquette TN la case malade-malade (la p. 40 corrige en TP) ; la p. 40 écrit le dénominateur de l'accuracy $TP + FN + FP + FN$ (lire $TP + FN + FP + TN$, le calcul 13 500/20 000 est juste) ; la p. 39 écrit $(1 - y_i)(1 - \log(1 - p(x_i)))$ là où la forme usuelle est $(1 - y_i)\log(1 - p(x_i))$.
- Le quiz 05.1 (perte calculée en $\log_2$) relève de cette formule ; son énoncé n'est pas dans le dépôt.

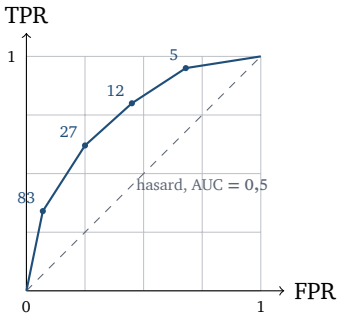
L'accuracy peut tromper (p. 41). Méthode A testée sur 10 000 hommes dont 40 % malades : $TP = 4000$, $FN = 0$, $FP = 2000$, $TN = 4000$, accuracy 0,8. Méthode B annoncée à 92 % sur 10 000 personnes. Si le groupe B comptait 8000 femmes et 2000 hommes (800 malades) et que B répond toujours « négatif » : $TP = 0$, $FN = 800$, $FP = 0$, $TN = 9200$, accuracy 0,92, sensibilité 0. B est inutile. Une accuracy ne se compare qu'à proportions de classes égales.

26. Courbe ROC et AUC

Source : `03_Classification`, p. 42 à 51 ; AUC de l'exemple calculée par `calc_embai_s36.py`

Un paramètre scalaire p classe en classe 1 si $p <$ seuil, en classe 2 sinon (p. 42). La **courbe ROC** trace la sensibilité (TPR) en fonction de 1 − spécificité (FPR) quand le seuil parcourt toutes les valeurs. L'**AUC** est l'aire sous cette courbe, entre 0 et 1.

Seuil (malade si $x \geq$)	−100	5	12	27	83	120
TPR (sensibilité)	1	0,95	0,80	0,62	0,34	0
FPR (1− spécificité)	1	0,68	0,45	0,25	0,07	0



À retenir

- Seuil très bas : tout le monde est malade, point $(1, 1)$. Seuil très haut : personne, point $(0, 0)$. Monter le seuil fait baisser TPR et FPR ensemble.
- **Attribut sans rapport** avec la classe (tirage au hasard) : la diagonale, AUC $= 0{,}5$ (p. 50, quiz 03.6). **Attribut idéal** : AUC $= 1$. AUC $< 0{,}5$: pire que le hasard. Une AUC utile est $> 0{,}5$.
- AUC par la méthode des trapèzes [rappel général] :

$$\text{AUC} \approx \sum_i \frac{(\text{FPR}_{i+1} - \text{FPR}_i)(\text{TPR}_i + \text{TPR}_{i+1})}{2}.$$

Sur l'exemple : $0{,}0119 + 0{,}0864 + 0{,}142 + 0{,}2013 + 0{,}312 = 0{,}754$.

Piège

La p. 49 écrit « AUC : 0…1, should be $< 0{,}5$ » ; la p. 50 dit « should be $> 0{,}5$ ». La bonne lecture est $> 0{,}5$. Sur le schéma de la p. 49 (deux cloches, normal et malade, seuils A à E), le seuil A, tout à gauche, déclare tout le monde malade : point en haut à droite de la courbe ; le seuil E, tout à droite, ne déclare personne : origine.

27. Clustering : objectif et préparation

Source : `04_Clustering`, p. 2 et 3

Le clustering regroupe des données **sans étiquette** (apprentissage non supervisé). Méthodes du cours : k-means, clustering hiérarchique, DBSCAN ; puis réduction de dimension par PCA (p. 2). Étapes (p. 3) :

1. Fixer le but : exploration, segmentation de clients, détection d'anomalies…
2. Collecter, nettoyer (bruit, valeurs aberrantes), choisir les variables utiles.
3. **Mettre à l'échelle** (min-max ou z-score) pour que chaque variable pèse autant.
4. Transformer si besoin, par exemple réduire la dimension par PCA.

Il faut une **distance** (similarité ou dissimilarité) et une **distance entre clusters** définie à partir d'elle.

28. Distances

Source : `04_Clustering`, p. 4 à 7 ; distances de Mahalanobis de l'exemple calculées par `calc_embai_s36.py`

Distance	Formule	Usage (slide)
Euclidienne (L_2)	$\sqrt{\sum_i (x_i - y_i)^2}$	la plus courante pour k-means
Manhattan (L_1)	$\sum_i \lvert x_i - y_i \rvert$	données de grande dimension
Minkowski	$\left(\sum_i \lvert x_i - y_i \rvert^p\right)^{1/p}$	$p = 2$ euclidienne, $p = 1$ Manhattan
Cosinus (similarité)	$\frac{A \cdot B}{\lVert A \rVert \lVert B \rVert}$	texte
Hamming	nombre de positions qui diffèrent ($\sum a_i \oplus b_i$)	données binaires et catégorielles
Mahalanobis (cas 2D)	$\sqrt{\frac{(x_M - x_P)^2}{\sigma_1^2} + \frac{(y_M - y_P)^2}{\sigma_2^2}}$	données continues, tient compte de la variance

Exemple de Hamming (p. 5) : 101100010100 et 001000110110 diffèrent en 4 positions. La Mahalanobis mesure la distance entre un point et une *distribution*, en tenant compte de sa variance et de ses corrélations ; la formule de la slide est le cas particulier 2D sans corrélation. [rappel général] Forme générale : $\sqrt{(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})}$.

Masse et pH sanguin, p. 7

Masse de 40 à 150 kg, pH de 7,34 à 7,45, distributions uniformes : moyennes 95 kg et 7,395. Quel point est le plus proche de la moyenne, $P_1 = (90;\ 7{,}395)$ ou $P_2 = (90{,}1;\ 7{,}45)$? Euclidienne : $d(M,P_1) = 5$, $d(M,P_2) = \sqrt{4{,}9^2 + 0{,}055^2} = 4{,}90031$. Elle désigne P_2, alors que son pH est à la limite de la plage : « la distance euclidienne n'est pas utile ici » (ni Minkowski, ni Manhattan).
[rappel général] Écart-type d'une loi uniforme sur $[a,b]$: $(b-a)/\sqrt{12}$, soit $\sigma_1 = 31{,}75$ kg et $\sigma_2 = 0{,}0318$. Mahalanobis : $d(M,P_1) = 5/31{,}75 = 0{,}157$; $d(M,P_2) = \sqrt{(4{,}9/31{,}75)^2 + (0{,}055/0{,}0318)^2} = 1{,}739$. P_1 est bien le plus proche.

Piège

Variables d'échelles très différentes : sans mise à l'échelle ni Mahalanobis, la variable de plus grande amplitude écrase les autres. C'est le sens probable du quiz 04.1 (« quelle distance utiliser ? ») et de l'item VRAI/FAUX sur Mahalanobis de la série 1.

29. K-means

Source : `04_Clustering`, p. 8 à 14

À retenir

1. Choisir le nombre de clusters K ; initialiser K centres (au hasard).
2. **Affectation** : chaque point va au centre le plus proche.
3. **Mise à jour** : chaque centre devient la moyenne des points qui lui sont affectés.
4. Répéter 2 et 3 jusqu'à convergence ou pendant un nombre d'itérations fixé.

Coût : $O(N \cdot K \cdot J \cdot d)$ (N points, K clusters, J itérations, d variables). La partie critique : NKd pour les affectations, *à chaque itération* (kNN : $O(Nd)$ par point à classer).

Arrêt (p. 9). $j > J_{\text{limite}}$, ou nombre de points qui changent de cluster dans l'itération inférieur à un seuil. **Sensibilité aux paramètres** : K et la distance.

Avantages (p. 8) : simple, souvent rapide. **Inconvénients** : K à fixer d'avance ; sensible aux centres initiaux ; suppose des clusters sphériques et de tailles voisines ; peut rester bloqué dans un optimum local.

Piège

Quiz 04.2, raté : « dans quel sens les centres bougent-ils au pas suivant ? ». Chaque centre va vers la moyenne des points qui lui sont *actuellement* affectés : vecteur déplacement = moyenne du groupe − ancien centre. Refaire d'abord l'affectation avec les centres actuels, puis calculer les moyennes. Exercices : `fiche-midterm-practice.pdf` (série 1, exercice 4, variante V1) et `entrainement-embai.pdf`, niveau 15.

Yinyang k-means, pour l'embarqué (p. 10 à 14)

Les NK distances par itération peuvent dépasser les ressources d'un système embarqué. Yinyang k-means évite la plupart de ces calculs grâce à l'inégalité triangulaire $d(A,C) \leq d(A,B) + d(B,C)$, d'où $d(A,C) - d(A,B) \leq d(B,C)$.

- On garde une borne supérieure $ub(x)$ de la distance de x à son centre $b(x)$. Quand ce centre se déplace en $b(x)'$: $ub(x)' = ub(x) + d(b(x), b(x)')$.
- On garde une borne inférieure $lb(x, c_k)$ de la distance aux autres centres. Quand c_k se déplace : $lb(x, c'_k) = lb(x, c_k) - d(c_k, c'_k)$.
- Si $ub(x)' < lb(x, c'_k)$, c'_k ne peut pas devenir le centre le plus proche de x : inutile de calculer $d(x, c'_k)$.

Par itération, il ne reste à calculer que les K déplacements de centres, plus quelques distances pour les points où le test échoue. Les centres bougent en général peu, d'où de grosses économies. D'autres étapes (regroupement des centres) ne sont pas détaillées en cours.

30. Clustering hiérarchique

Source : `04_Clustering`, p. 15 à 30

> **À retenir**
>
> **Ascendant (agglomératif, p. 15).** Au départ, N points = N clusters. À chaque étape, on fusionne les deux clusters les plus proches : le nombre de clusters baisse de 1. On s'arrête à un seul cluster ou sur une condition d'arrêt. Le **dendrogramme** représente les fusions : feuilles = points, racine = cluster unique, hauteur d'une fusion = distance à laquelle elle a eu lieu.

Avec une distance limite (p. 16 à 23). Choisir les variables, leur poids et la distance (euclidienne, Manhattan…). Deux éléments ne peuvent être dans le même cluster que si leur distance est inférieure à la limite. On fusionne tant que la plus petite distance est sous la limite. Les points restés seuls, trop loin de tout cluster, sont des **anomalies** (outliers) : la p. 23 en montre 5, à côté de 3 clusters.

Lire le dendrogramme (p. 24 et 30). Tracer l'horizontale à la distance limite et compter les traits verticaux coupés. Sur la figure de la p. 24, l'horizontale « chosen distance limit » coupe trois branches : $\{p_6, p_{10}, p_1, p_{11}\}$, $\{p_4, p_{12}, p_2, p_{13}, p_{14}\}$ et $\{p_3, p_{15}, p_5, p_8, p_9, p_7\}$ (lecture de la figure). Exercice corrigé sur une figure de ce type : `fiche-midterm-practice.pdf`, série 2, exercice 2.

Distance entre clusters (p. 26 et 27).

Critère	Formule
Maximum (complete linkage)	$\max\{d(a,b) : a \in A, b \in B\}$
Minimum (single linkage)	$\min\{d(a,b) : a \in A, b \in B\}$
Moyenne (average linkage)	$\frac{1}{\lvert A\rvert \lvert B\rvert} \sum_{a\in A} \sum_{b\in B} d(a,b)$
Centroïde	$\lVert c_A - c_B \rVert$, c centre du cluster

Sur la figure de la p. 27, le centroïde fait fusionner 1 et 3, le single linkage 1 et 2, le complete linkage 1 et 3 (la slide écrit $d_{S13} < d_{S12}$ pour le maximum : lire $d_{M13} < d_{M12}$) : les critères **peuvent donner des résultats différents** (quiz 04.3 et 04.4, « quelles deux fusionner ? », « le résultat final sera-t-il le

même ? »).

Avantages (p. 28)	Inconvénients (p. 28)
pas de nombre de clusters à fixer d'avance (on coupe le dendrogramme) ; représentation en arbre parlante	une fusion ne se défait pas ; plus coûteux que k-means : $O(n^3)$ en force brute, $O(n^2 \log n)$ pour le single linkage avec un bon algorithme

Descendant (divisif, p. 29). Tout l'ensemble est un cluster, qu'on coupe en deux à chaque étape jusqu'à un point par cluster ou une condition d'arrêt. Peut tenir compte de propriétés globales ; plus coûteux que l'agglomératif ; une coupe ne se défait pas non plus. Les deux approches dépendent du choix de la distance et de la distance limite (p. 30).

31. DBSCAN et données manquantes

Source : `04_Clustering`, p. 31 à 39

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) : un cluster est une région dense, séparée des autres par des régions peu denses ; les zones clairsemées sont du bruit. Hyperparamètres : ε (deux points à moins de ε sont voisins) et *MinPts* (nombre minimal de voisins, le point compris, pour être un point cœur).

- **Point cœur** : au moins *MinPts* points dans son ε-voisinage. **Point frontière** : moins de *MinPts* voisins, mais dans le voisinage d'un point cœur. **Bruit** : ni l'un ni l'autre (p. 33).
- Algorithme (p. 32) : partir d'un point non visité ; s'il est cœur, ouvrir un cluster, y ajouter ses voisins, et étendre par les voisins qui sont eux-mêmes cœurs ; recommencer jusqu'à ce que tout point soit dans un cluster ou marqué bruit.
- Avantages (p. 34) : clusters de forme quelconque, bruit isolé, pas de K à fixer. Inconvénients : $O(n^2)$, densités variables mal gérées, sensible à ε et *MinPts*.

Estimer les données manquantes (p. 35 à 39). Les exemples sont coûteux à collecter : on préfère estimer une valeur manquante qu'écarter l'exemple.

Dans un projet industriel de l'enseignant, plus de 10 % des exemples avaient une valeur manquante, et le modèle final était meilleur en les estimant. Démo 1 : $x_2 = 8$ connu, x_1 manquant, on prend la valeur de x_1 la plus probable sachant x_2. Démo 2 : image dont 20 % des pixels ont perdu une composante R, V ou B ; la correction par moyennes du cluster (déterminé avec les deux composantes restantes) fait mieux que la correction par moyennes globales.

32. Malédiction de la dimension

Source : `04_Clustering`, p. 40 à 42

Les méthodes d'apprentissage sont statistiques : elles comptent des observations dans des régions de l'espace (arbres, réseaux, kNN). Le nombre d'observations nécessaires croît comme $O(\text{const}^{\text{dim}})$. Exemple de la p. 40 : 20 cases par variable (pas de 5 %) donnent $20^2 = 400$, $20^3 = 8000$, $20^4 = 160\,000$ cases, et $20^{12} \approx 4{,}1 \cdot 10^{15}$ pour 12 attributs.

Conséquences (p. 41) : données clairsemées et risque de surapprentissage ; distances toutes à peu près égales, ce qui gêne k-means et kNN ; calcul et stockage qui explosent ; perte d'intuition au-delà de 3 dimensions ; redondance des variables corrélées. Buts de la réduction de dimension (p. 42) : visualisation, compression, réduction du bruit, analyse de tendance, détection d'anomalies.

33. Analyse en composantes principales (PCA)

Source : `04_Clustering`, p. 43 à 50

Méthode fondée sur les données, qui donne un repère hiérarchisé : la première direction est celle où la variance est maximale (p. 43).

À retenir

1. **Centrer** : $\mathbf{X}$ a N lignes (observations, vecteurs *lignes* $\mathbf{x}_i$) et n colonnes. On retire à chaque colonne sa moyenne $\mu_j = \frac{1}{N}\sum_i x_{ij}$: $\mathbf{X}_{(st)}$ (p. 44).
2. **Covariance** : on cherche $\mathbf{u}$ unitaire qui maximise la variance des projections $\frac{1}{N}\sum_i (\mathbf{x}_{(st)i}\mathbf{u})^2 = \mathbf{u}^\top \mathbf{\Sigma} \mathbf{u}$, avec $\mathbf{\Sigma} = \frac{1}{N}\sum_i \mathbf{x}_i^\top \mathbf{x}_i$. Solution : le vecteur propre de la plus grande valeur propre, et $\mathbf{u}^\top \mathbf{\Sigma} \mathbf{u} = \lambda_{\max}$ (p. 45).
3. **Valeurs propres** λ_i et vecteurs propres $\mathbf{u}_i$ de $\mathbf{\Sigma}$, rangés par ordre décroissant ; $\mathbf{\Sigma}$ est symétrique, ses vecteurs propres sont orthogonaux (p. 46).
4. **Choisir** r **composantes** d'après la variance décrite $\frac{\sum_{i=1}^{r} \lambda_i}{\sum_i \lambda_i}$ (p. 47).
5. **Projeter** : $\mathbf{Y} = \mathbf{U}^\top \mathbf{X}_{(st)}^\top$ avec $\mathbf{U} = [\mathbf{u}_1 \dots \mathbf{u}_r]$ (p. 47).

Avantages (p. 48)	Inconvénients (p. 48)
réduction de dimension, visualisation, sélection de variables, variables très corrélées, compression, détection d'anomalies	nouvelles variables difficiles à interpréter ; relations **linéaires** seulement

Démo (p. 49 et 50). Nuage masse (kg), taille (cm) très corrélé ; première composante NewParam $\approx$ BodyMass + (Height − 195)/3 − 73 : « difficile à interpréter ».

À la main, 2 variables

Points $(7,5)$, $(1,-1)$, $(5,1)$, $(3,3)$. Moyenne $(4,2)$; centrés : $(3,3)$,

$(-3,-3)$, $(1,-1)$, $(-1,1)$. $\mathbf{\Sigma} = \frac{1}{4}\begin{pmatrix} 20 & 16 \\ 16 & 20 \end{pmatrix} = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}$. [rappel général] Valeurs propres d'une 2×2 : racines de $\lambda^2 - \mathrm{tr}\,\lambda + \det = 0$, ici $\lambda^2 - 10\lambda + 9 = 0$, donc $\lambda_1 = 9$, $\lambda_2 = 1$. Vecteur propre de 9 : $(1,1)/\sqrt{2}$. Variance décrite par la première composante : $9/10 = 90$ %. Projections sur $\mathbf{u}_1$: $\pm 6/\sqrt{2} = \pm 4{,}243$ et 0, 0.

34. TP 3 : PCA par SVD, compression d'image

Source : `gyak03-pca-svd/` : `pca_iris.m`, `pca_cancer.m`, `svd_picture.m` ; valeurs iris calculées sur `irisdata.txt` par `calc_embai_s36.py`

Les trois scripts MATLAB passent par la **décomposition en valeurs singulières** `[U,S,V] = svd(X)`, soit $\mathbf{X} = \mathbf{USV}^\top$.

- `pca_iris.m` : centre les données (`bsxfun(@minus, X, mean(X))`), calcule la SVD, garde `NComp = 3` colonnes de $\mathbf{V}$ (« principal components ») et projette : `XV = X*Vreduced`. Il trace les valeurs singulières (`stem(diag(S))`) et la variance cumulée `cumsum(diag(S).^2)./sum(diag(S).^2)`.
- `pca_cancer.m` : même démarche sur les données de cancer de l'ovaire, 3 composantes, nuage 3D coloré « Normal » et « Cancer ».
- `svd_picture.m` : image en niveaux de gris, SVD, puis reconstruction $\mathbf{US}_{\text{new}}\mathbf{V}^\top$ où $\mathbf{S}_{\text{new}}$ ne garde que les `NComp` premières valeurs singulières (les autres mises à 0).

À retenir

[rappel général] Lien avec le cours : si $\mathbf{X}$ est centrée, les colonnes de $\mathbf{V}$ sont les vecteurs propres de $\mathbf{\Sigma}$, et $\lambda_i = s_i^2/N$ (avec la convention $\frac{1}{N}$ de la p. 45). La variance cumulée du TP, $\sum_{i\le r} s_i^2 / \sum_i s_i^2$, est donc exactement la « variance décrite » de la p. 47. Sur l'exemple à la main ci-dessus : $s = (6,2)$, $36/40 = 0{,}9$.

Iris, recalculé sur `irisdata.txt`

150 fleurs, 4 mesures. Valeurs singulières des données centrées : 25,09 ; 6,01 ; 3,42 ; 1,88. Variance cumulée : 92,5 % ; 97,8 % ; 99,5 % ; 100 %.

Une composante décrit déjà 92 % de la variance ; deux suffisent pour un nuage 2D lisible.

Piège

- `pca_cancer.m` **ne centre pas X** avant la SVD : la première « composante » capte alors surtout la moyenne. Pour une vraie PCA, centrer d'abord (étape 1 de la p. 44).
- `pca_iris.m`, figure 3 : les tracés de versicolor et virginica utilisent `y(species=="setosa")` et `z(species=="setosa")`, une erreur de copier-coller.
- [rappel général] Compression : garder r valeurs singulières d'une image $m \times n$ demande de stocker $r(m + n + 1)$ nombres au lieu de mn.

35. Cours 5 : détection de chute sur un système portable

Source : `05_UseCase`, p. 1 à 17 ; métriques calculées par `calc_embai_s36.py`

Cas d'usage tiré d'un article publié chez MDPI (`mdpi.com/1999-5903/18/4/211`, p. 1). Les chutes sont la première cause de blessure des personnes âgées : blessures légères dans 30 à 50 % des cas, graves dans 10 %, fracture de la hanche dans 1 % ; aux États-Unis, environ 800 000 hospitalisations par an et 300 000 fractures de la hanche ; incidence annuelle jusqu'à 50 % après 80 ans. Population de 80 ans et plus : 476 749 en Hongrie (4,93 %) (p. 2).

Proposition (p. 4). Un modèle TinyML et une communication IoT pour surveiller et détecter les chutes en temps réel : capteurs inertiels (IMU : accéléromètre, gyroscope, magnétomètre) ; classifieur à trois classes, chute, marche, repos (falling, walking, idle) ; optimisé pour microcontrôleur, en virgule flottante et en version quantifiée ; inférence sur l'appareil ; évaluation de la mémoire, de la latence et de la faisabilité.

Chaîne de traitement (p. 5). Acquisition de l'accéléromètre ; normalisation ; segmentation en fenêtres de longueur fixe qui se recouvrent ; extraction de caractéristiques ; classifieur TinyML ; prédiction en temps réel.

À retenir

- **Données (p. 6)** : a_x, a_y, a_z à 62,5 Hz, complétés par des zéros (zero padding) pour une longueur uniforme ; essais de 2 s, environ 2 h au total, soit environ 3600 essais ; étiquetage manuel ; découpage 70 : 15 : 15. Calcul : 125 échantillons par axe et par fenêtre (375 valeurs), et 2520/540/540 essais.
- **Normalisation (p. 10)** : $x_{nk} = \frac{x_k - \text{mean}(X)}{\text{std}(X)}$, avec $\text{std}(X) = \sqrt{\frac{1}{N}\sum_k (x_k - \text{mean}(X))^2}$ (division par N).
- **Caractéristiques (p. 12 et 13)** : norme $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$; G-force $G_t = |a_t|/9{,}8$; inclinaison $\cos\theta = \frac{\mathbf{a}_{t_1} \cdot \mathbf{a}_{t_2}}{|\mathbf{a}_{t_1}||\mathbf{a}_{t_2}|}$; vitesse $V_t = V_{t_0} + \sum_k a_k\, dt$; écart-type, asymétrie (skewness), aplatissement (kurtosis) ; durées de chute, d'activité, de transition, de récupération après

la chute.

- **Classifieur (p. 14)** : petit réseau de neurones à propagation avant (TinyML), calcul en float32 puis en int8.
- **Détection d'anomalie (p. 16)** : k-means sur des vecteurs de 9 caractéristiques normalisées ; chaque cluster a un centre c_k et un rayon r_k. Score $d(\mathbf{x}) = \min_k(\|\mathbf{x} - c_k\|_2 - r_k)$: négatif ou proche de 0, normal ; au-dessus d'un seuil, anomalie.

Matrices de confusion (p. 15), lignes = réel, colonnes = prédit, ordre chute, repos, marche :

float32	chute	repos	marche	int8	chute	repos	marche
chute	187	0	0	chute	187	0	0
repos	2	183	2	repos	3	181	3
marche	0	2	185	marche	0	3	184

Calculs : 561 fenêtres de test (187 par classe). Accuracy float32 555/561 = 98,9 % ; int8 552/561 = 98,4 %. Sensibilité de la classe chute : 1 dans les deux cas (aucune chute manquée) ; précision de la classe chute 187/189 = 0,989 puis 187/190 = 0,984 (fausses alertes venant du repos). La quantification coûte 3 bonnes réponses sur 561 et divise la mémoire des poids par 4 (concept 13).

p. 17 : nuage des données d'entraînement (bleu) et de test « walking » (orange) dans le plan des RMS de a_x et a_y, avec les sphères des clusters ; scores d'anomalie du test entre −0,5616 et −0,0707 (moyenne −0,3347) : tous négatifs, aucune anomalie.

Piège

Ce cours réunit trois thèmes du midterm : matrice de confusion à 3 classes, k-means, contraintes de l'embarqué (int8). Dans une matrice multiclasse, la sensibilité d'une classe se lit sur sa *ligne*, la précision sur sa *colonne*. Fiche courte du même cours : `fiche-midterm-practice.pdf`, dernier concept avant les variantes.

36. Réseaux de neurones : sources utilisées

Source : fiche `CLAUDE.md` de la matière, « État du matériel »

> **Piège**
>
> **Les slides du cours 6 (réseaux de neurones, quiz 06.1 à 06.3) ne sont pas dans le dépôt** au 2026-10-17 : absentes des exports OneDrive du 15 et du 17 octobre. Cette partie s'appuie sur les **supports du TP 4** : `NN2a` (modèle de neurone, architectures, apprentissage, 48 slides), `NN4a` (rétropropagation, 51 slides) et, pour la généralisation, `NN8` et `NN1` du dépôt `Fundamentals-of-Neural-Networks` (Primož Potočnik, université de Ljubljana, 2022). Les notations peuvent différer de celles de l'enseignant au cours 6, en particulier la fonction d'erreur ($E = \frac{1}{2}e^2$ en TP, $E = \varepsilon^2$ dans les annales). Les numéros cités sont ceux des slides PowerPoint.

Au programme du midterm (fiche de la matière) : introduction aux réseaux de neurones, impact du bruit sur l'apprentissage, surapprentissage, early stopping, backtracking, découpage entraînement, validation, test. D'après l'enseignant, un calcul du type quiz 06.3 (nouvelle valeur d'un poids après un pas d'apprentissage) peut tomber : c'est la méthode qu'il faut savoir (concept 41).

37. Le neurone

Source : `NN2a`, slides 2, 3, 6 et 7 ; `NN1`, slides 11 et 14

> **À retenir**
>
> Neurone à entrée vectorielle : entrées $\mathbf{x} = [x_1, \dots, x_R]$, poids $\mathbf{w} = [w_1, \dots, w_R]$, biais b. Potentiel d'activation $v = \mathbf{w}\mathbf{x} + b = w_1 x_1 + \cdots + w_R x_R + b$; sortie $y = f(v)$, f la fonction d'activation. Paramètres ajustables : les poids et le biais (slides 2 et 3).

Le biais se traite comme un poids w_0 porté par une entrée constante $x_0 = 1$ (notation des annales et de `NN4a`, slide 10, où la somme commence à $i = 0$). Notation MATLAB (slide 7) : entrée $\mathbf{p}$ de taille $R \times 1$, poids $\mathbf{W}$ de taille $1 \times R$, produit scalaire $\mathbf{Wp}$.

Neurone de McCulloch et Pitts (1943, slide 6). Entrée vectorielle, activation à seuil : $y = 1$ si $\mathbf{wx} \geq -b$, 0 sinon. Modèle très simplifié du neurone biologique (sortie binaire, somme linéaire, pas de temps ni d'aléa), mais un réseau de tels neurones peut calculer toute fonction calculable.

Cerveau et machine (NN1, slide 11). Environ 10^{10} neurones, 10^4 synapses par neurone, environ 1 ms de temps de traitement par neurone : un fonctionnement lent, compensé par un nombre énorme d'unités et de connexions et un parallélisme massif. Le cours 2 donne 20 W pour le cerveau (concept 8). Ce qui reste difficile pour un réseau (slide 14) : prédire des nombres aléatoires, factoriser de grands entiers, tester la primalité, déchiffrer un bon chiffrement.

38. Fonctions d'activation

Source : NN2a, slides 4 et 5 ; NN4a, slides 4, 23 et 27 ; valeurs calculées par `calc_embai_s36.py`

Fonction (nom MATLAB)	Définition	Dérivée
Seuil (`hardlim`)	1 si $v \geq 0$, 0 sinon	nulle (non dérivable en 0)
Linéaire (`purelin`)	v	1
Logistique (`logsig`)	$\frac{1}{1+e^{-av}}$, valeurs dans $]0,1[$	$a\,y(1-y)$
Tangente hyperbolique (`tansig`)	$\tanh v$, valeurs dans $]-1,1[$	$1-y^2$ [rappel général]
`satlin`	0 si $v<0$, v sur $[0,1]$, 1 si $v>1$	1 entre 0 et 1, 0 ailleurs
`satlins`	-1, v sur $[-1,1]$, 1	1 entre -1 et 1, 0 ailleurs
Base radiale (`radbas`)	cloche de maximum 1 en 0, 0,5 en $\pm 0{,}833$	
`softmax`	$\frac{e^{n_i}}{\sum_j e^{n_j}}$ [rappel général]	

- La dérivée de la logistique s'écrit avec la seule sortie : $f'(v) = a\,y(1-y)$ (NN4a, slide 23), d'où un gradient local calculable sans réévaluer f. Avec $a = 1$: $\sigma'(0) = 0{,}25$, $\sigma'(\pm 2) = 0{,}105$, $\sigma'(\pm 5) = 0{,}0066$.

- La rétropropagation exige une activation **dérivable** : le seuil dur du perceptron ne convient pas (slide 4) ; sans non-linéarité, un réseau multicouche n'est pas plus puissant qu'une seule couche (slide 4).
- Une sigmoïde **antisymétrique** accélère l'apprentissage ; choix courant $f(v) = a\tanh(bv)$ avec $a = 1{,}72$, $b = 0{,}67$: $f(\pm 1) = \pm 1$ et pente effective proche de 1 en 0 ($1{,}72 \times 0{,}67 = 1{,}15$) (slide 27).
- Exemple `softmax` de la slide 5 : entrée (0; 1; −0,5; 0,5), sortie (0,17; 0,46; 0,10; 0,28), de somme 1. Recalcul : $e^0 = 1$, $e^1 = 2{,}718$, $e^{-0{,}5} = 0{,}607$, $e^{0{,}5} = 1{,}649$, somme 5,973.

> **Piège**
>
> **ReLU** ($\max(0, v)$, dérivée 1 pour $v > 0$, 0 pour $v < 0$) n'apparaît pas dans les supports de TP ; la série d'entraînement 1 demande pourtant de la dessiner avec la sigmoïde et tanh, et d'expliquer le **vanishing gradient**. Courbes et réponse type : `fiche-midterm-practice.pdf`, concept « Fonctions d'activation et vanishing gradient » (rappel général). Côté supports : `NN4a`, slide 33, « paralysie du réseau » : sigmoïdes et très grands poids rendent les gradients presque nuls (*vanishing gradient problem*), l'apprentissage s'arrête presque ; slide 37 : la pente de la sigmoïde tend vers 0 pour de grandes entrées, d'où la rétropropagation résiliente (Rprop) qui n'utilise que le signe de la dérivée.

39. Architectures et règles d'apprentissage

Source : `NN2a`, slides 8 à 23 et 44 ; `NN4a`, slides 2 et 3

Architectures (`NN2a`, slides 8 à 13).

- **Une couche, propagation avant** : les entrées se projettent sur une couche de neurones de sortie, la seule qui calcule.
- **Multicouche, propagation avant** (perceptron multicouche, MLP) : une ou plusieurs couches cachées, qui extraient des statistiques d'ordre supérieur. Flux strictement de l'entrée vers la sortie, pas de rétroaction : réseau statique, apprentissage facile. La couche d'entrée ne fait que distribuer (`NN4a`, slide 3).
- **Récurrent** (« dynamique ») : au moins une boucle de rétroaction avec

retard ; la sortie dépend aussi des entrées, sorties ou états précédents. Bonnes capacités temporelles.

Motivation de la rétropropagation (`NN4a`, slide 2) : une couche ne résout que les problèmes linéairement séparables (Minsky et Papert, 1969) ; l'entraînement des réseaux multicouches est résolu par rétropropagation de l'erreur (Werbos 1974 ; Rumelhart 1986). XOR est l'exemple type (live script `NN4b`).

Règles d'apprentissage (`NN2a`, slides 14 à 19).

Règle	Principe
Correction d'erreur	$e(t) = d(t) - y(t)$; on minimise l'énergie instantanée $\xi(t) = \frac{1}{2}e^2(t)$. Règle delta (Widrow-Hoff) : $\Delta w(t) = \eta\, e(t)\, x(t)$, correction proportionnelle à l'erreur et à l'entrée. Le pas η règle la stabilité (boucle fermée)
À mémoire	exemples stockés ; voisinage du nouveau point : plus proche voisin, kNN, réseaux RBF
Hebb (1949)	renforcer une synapse dont les deux neurones s'activent ensemble : $\Delta w(t) = \eta\, y(t)\, x(t)$
Compétitive	un seul neurone de sortie gagne (le plus grand potentiel) et rapproche ses poids de l'entrée

Paradigmes (slides 20 à 23). Supervisé (exemples entrée, sortie ; correction d'erreur, à mémoire) ; non supervisé (pas de professeur ; compression, réduction de dimension, clustering ; Hebb, compétitif) ; par renforcement (signal de renforcement retardé ; un réseau « critique » le transforme en signal heuristique).

Découpage des données (slide 44). Validation croisée : entraînement 80 %, test 20 % ; l'entraînement se recoupe en apprentissage 60 % et validation 20 %. Validation pour régler les hyperparamètres ; test, à la fin, pour estimer la généralisation sur des données jamais vues. Le cas d'usage du cours 5 découpe en 70 : 15 : 15.

40. Rétropropagation

Source : NN4a, slides 6 à 17 (formules lues dans les objets équation des slides)

Deux passes (slide 6). Passe avant : l'entrée se propage jusqu'à la sortie, poids figés, on calcule l'erreur. Passe arrière : l'erreur remonte, on calcule les gradients, on ajuste les poids.

À retenir

Notations : y_i entrée du neurone j (sortie de la couche précédente, ou entrée x_i), w_{ji} poids de i vers j, $v_j = \sum_{i=0} w_{ji} y_i$ (avec $y_0 = 1$ pour le biais), $y_j = f(v_j)$, $e_j = d_j - y_j$.

- Énergie d'erreur instantanée (slide 7) : $E(n) = \frac{1}{2} \sum_{j=1}^{R} e_j^2(n)$; moyenne sur l'ensemble : $\bar{E} = \frac{1}{N} \sum_n E(n)$. Mise à jour après chaque exemple (pattern by pattern, slide 8).
- Dérivation en chaîne (slides 9 à 11) : $\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial e_j} \frac{\partial e_j}{\partial y_j} \frac{\partial y_j}{\partial v_j} \frac{\partial v_j}{\partial w_{ji}} = e_j \cdot (-1) \cdot f'(v_j) \cdot y_i$.
- **Règle delta** : $\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}} = \eta \, \delta_j \, y_i$: correction = pas × gradient local × entrée du neurone j (slide 15).
- Gradient local d'un neurone de **sortie** : $\delta_k = e_k \, f'(v_k)$.
- Gradient local d'un neurone **caché** : $\delta_j = f'(v_j) \sum_k \delta_k \, w_{kj}$ (l'erreur n'est pas mesurable sur un neurone caché : on rétropropage, slides 12 à 14).
- Logistique : $f'(v_j) = a \, y_j (1 - y_j)$.

ADALINE (slide 20). Pas de couche cachée, une sortie, activation linéaire : $f' = 1$, $\delta = e$, et la rétropropagation redonne la règle delta d'origine $\Delta w_i = \eta \, e \, x_i$. La rétropropagation généralise la règle delta (LMS).

Résumé de l'algorithme (slide 17). Initialiser les poids (loi uniforme centrée, variance qui place les potentiels entre la partie linéaire et la partie saturée de la sigmoïde) ; pour chaque exemple : passe avant, passe arrière, mise à jour ; itérer jusqu'au critère d'arrêt.

Piège

- **Convention de l'erreur.** Le TP pose $E = \frac{1}{2}e^2$, donc $\delta = e\,f'(v)$. Les corrigés des annales de l'enseignant prennent $E = \varepsilon^2$, donc un facteur 2 : $\delta = 2\varepsilon f'(v)$ (`exams/annales-pataki-analyse.md`, section 6.4). Convention du cours 6 : **À VÉRIFIER** (slides absentes). À l'examen, écrire la fonction d'erreur en première ligne ; si l'énoncé la donne, l'appliquer telle quelle.
- **Signe.** $e = d - y$ et $\Delta w = +\eta\,\delta\,y_i$; si l'on écrit $e = y - d$, le signe de la mise à jour s'inverse. Contrôle : si $y < d$ et que l'entrée et f' sont positives, le poids doit monter.
- **Quiz 06.1** (« pourquoi pas $E = |\varepsilon|$ ou $E = \varepsilon$? ») [rappel général] : avec $E = \varepsilon$, minimiser pousserait ε vers $-\infty$ et des erreurs de signes opposés se compenseraient ; $|\varepsilon|$ n'est pas dérivable en 0 et son gradient garde la même amplitude quelle que soit la taille de l'erreur ; ε^2 est dérivable partout et pèse plus sur les grandes erreurs.

41. Méthode : un pas d'apprentissage (type quiz 06.3)

Source : formules de `NN4a`, slides 7 à 15 ; exemple et calculs vérifiés par `calc_embai_s36.py`

À retenir

1. Écrire la convention : $E = \frac{1}{2}e^2$ (facteur $c = 1$) ou $E = e^2$ ($c = 2$), $e = d - y$.
2. **Passe avant**, neurone par neurone : $v = \sum_i w_i y_i$ (biais ×1 compris), puis $y = f(v)$. Garder 4 décimales.
3. Sortie : $\delta_{\text{out}} = c\,e\,f'(v_{\text{out}})$, avec $f' = y(1-y)$ pour la logistique.
4. Caché j : $\delta_j = f'(v_j)\,w_{\text{out},j}\,\delta_{\text{out}}$ (avec *l'ancien* poids de sortie ; une somme s'il y a plusieurs sorties).
5. Mise à jour : $w_{\text{nouveau}} = w + \eta\,\delta$ (entrée portée par ce poids) : 1 pour un biais, x_i pour un poids d'entrée, y_j pour un poids caché vers la sortie.

Un neurone logistique

$\mathbf{w} = (w_0, w_1, w_2) = (0{,}2;\ 0{,}5;\ -0{,}4)$, entrées $(1;\ 1;\ 2)$, cible $d = 1$, $\eta = 0{,}5$. $v = 0{,}2 + 0{,}5 - 0{,}8 = -0{,}1$; $y = \sigma(-0{,}1) = 0{,}4750$; $e = 0{,}5250$; $f'(v) = 0{,}4750 \times 0{,}5250 = 0{,}2494$. Avec $E = \frac{1}{2}e^2$: $\delta = 0{,}5250 \times 0{,}2494 = 0{,}1309$, $\Delta w_i = 0{,}5 \times 0{,}1309 \times x_i$, d'où $\mathbf{w}' = (0{,}2655;\ 0{,}5655;\ -0{,}2691)$. Avec $E = e^2$: $\delta = 0{,}2618$, $\mathbf{w}' = (0{,}3309;\ 0{,}6309;\ -0{,}1382)$. Dérivée demandée au quiz 06.2 sous cette forme : $\frac{\partial y}{\partial w_2} = f'(v)\, x_2 = 0{,}2494 \times 2 = 0{,}4988$.

Exercices : `fiche-midterm-practice.pdf` (série 1, exercice 2, réseau à deux couches, et variantes V4, V5) et `entrainement-embai.pdf`, niveau 20 (réseau 2-2-1 complet).

42. Entraîner en pratique

Source : `NN4a`, slides 21 à 44

- **Séquentiel ou par lot (slide 22).** Époque = présentation de tout l'ensemble. Par lot : mise à jour après chaque époque, plus précis mais très lent. Séquentiel : après chaque exemple, nature stochastique, convergence plus rapide, simple à coder : mode préféré, avec un ordre aléatoire des exemples.
- **Pas η (slide 25).** Le plus grand possible sans oscillations.
- **Arrêt (slide 26).** La convergence ne se démontre pas en général. Critères : norme du gradient assez petite ; erreur de sortie assez petite ou qui baisse trop peu par époque ; généralisation qui a atteint son maximum ; nombre maximal d'itérations.
- **Heuristiques (slides 27 à 29).** Exemples les plus informatifs, ordre aléatoire ; sigmoïde antisymétrique ; cibles dans la plage de l'activation avec une marge, par exemple $\max(\text{cible}) = 0{,}9 \max(f)$, sinon saturation ; entrées centrées, décorrélées (par PCA) et de variances voisines ; poids initiaux ni trop grands (saturation) ni trop petits (lenteur) : écart-type $\sigma_w = m^{-1/2}$ pour m poids d'entrée, avec $1{,}72 \tanh(0{,}67v)$.
- **Moment (slide 35).** $\Delta w_{ji}(n) = \alpha\, \Delta w_{ji}(n-1) + \eta\, \delta_j(n)\, y_i(n)$, $0 \le \alpha < 1$: accélère dans les descentes régulières, permet un pas plus grand sans instabilité.

- **Pas variable (slide 36)** : on accepte la mise à jour et on augmente η si l'erreur baisse ; on la rejette, on réduit η et on remet le moment à 0 si elle monte de plus de ζ. **Rprop (slide 37)** : seul le signe de la dérivée compte.
- **Limites (slide 33)** : entraînement long ; paralysie (vanishing gradient) ; minima locaux (remède : méthodes probabilistes, recuit simulé).
- **Algorithmes avancés (slides 38 à 44)** : Newton (inverse du hessien), quasi-Newton (BFGS, rapide pour les petits réseaux), gradient conjugué (grands réseaux, peu de mémoire), Levenberg-Marquardt (le plus rapide jusqu'à quelques centaines de poids ; `trainlm`, utilisé dans les live scripts).

43. Généralisation, bruit, early stopping

Source : `NN4a`, slides 30 à 32 et 45 à 48 ; `NN8`, slides 28 à 41 ; `03_Classification`, p. 28 à 32

> **À retenir**
>
> - **Généraliser** : répondre juste sur des données de test jamais utilisées à l'entraînement, tirées de la même population, et sur des entrées un peu différentes des exemples (`NN4a`, slide 30).
> - **Sous-apprentissage** : trop peu de neurones ou régularisation extrême. **Surapprentissage** : le réseau apprend parfaitement l'entraînement, y compris le bruit, et se trompe sur le test (`NN8`, slide 31).
> - **Effets de taille (`NN4a`, slides 46 à 48).** Plus d'exemples : erreur d'entraînement plus haute mais meilleure généralisation (4 exemples contre 20). Plus de neurones cachés : l'erreur d'entraînement baisse toujours, l'erreur de test baisse puis remonte (effet de pic) : il existe un nombre optimal.

Impact du bruit. Avec des données bruitées, on ne veut pas classer juste tous les exemples d'entraînement : une frontière ou une courbe qui ne passe pas par tous les points généralise mieux (`NN8`, slides 29 et 30 ; `03_Classification`, p. 29 et 30). À l'inverse, ajouter un petit bruit (jitter) aux *entrées* pendant l'entraînement « étale » les points et réduit le surapprentissage ; en ajouter aux cibles ne change pas les poids optimaux, cela ralentit seulement (`NN8`, slide 41).

Remèdes (`NN4a`, slide 31 ; `NN8`, slide 32). Réseau petit (moins de neurones

cachés, poids partagés) ; grand ensemble d'entraînement ; validation croisée ; early stopping ; régularisation ; bruit sur les entrées.

Early stopping et backtracking (`NN8`, slides 35 à 37). Trois ensembles : entraînement, validation (pour décider de l'arrêt quand son erreur se met à monter), test (estimation finale et comparaison des modèles). Procédure : beaucoup de neurones cachés, poids initiaux très petits, pas lent, erreur de validation calculée périodiquement, arrêt quand elle remonte. Comme l'erreur de validation peut monter et descendre plusieurs fois, le plus sûr est d'entraîner jusqu'à convergence en sauvegardant les poids à chaque époque, puis de **revenir aux poids de l'époque d'erreur de validation minimale**. Le K-fold (K sous-ensembles, K entraînements, moyenne) et le leave-one-out servent quand les données sont rares (slide 34).

Régularisation (`NN4a`, slide 32 ; `NN8`, slides 39 et 40). Fonction de performance $\text{msreg} = \gamma\,\text{mse} + (1-\gamma)\,\text{msw}$, avec $\text{mse} = \frac{1}{N}\sum(d-y)^2$ et $\text{msw} = \frac{1}{M}\sum w_m^2$: poids plus petits, réponse plus lisse, moins de surapprentissage. C'est le « weight decay », l'équivalent de la régression ridge du TP 2 (concept 17).

> **Piège**
>
> Validation ≠ test (`NN8`, slide 33). La validation sert pendant l'entraînement (arrêt, choix des hyperparamètres) ; le test ne sert qu'une fois, à la fin. Une erreur d'entraînement basse ne garantit rien sur la performance réelle (`NN4a`, slide 46).

44. Ce qui reste au programme

Source : `01_Intro`, slide 4 ; fiche `CLAUDE.md`, « Programme » et « Midterm »

Le midterm s'arrête aux réseaux de neurones (cours 6) inclus ; le cours de la semaine suivante n'est pas au programme (fiche de la matière, d'après Elisha, 2026-10-15). Restent pour l'examen final, sans support au 2026-10-17 :

1. Réseaux convolutifs ; système de reconnaissance de formes embarqué.
2. Réseaux récurrents et prédiction (les slides `NN5a` du TP 4, réseaux dynamiques, ne sont pas traitées ici).

3. Auto-encodeurs.
4. Apprentissage profond, réseaux génératifs, transformers.
5. Adaptation des réseaux de neurones à l'embarqué (quantification et élagage, annoncés en `01_Intro`, slide 28).
6. Lectures complémentaires, au programme de l'examen final.

Manquent aussi pour le midterm : les slides du cours 6 (concept 36) et les sujets des années précédentes, promis par l'enseignant le 2026-09-24. Les séries d'entraînement 2026 sont traitées dans `fiche_midterm_practice.pdf`.

45. Méthode d'examen

Source : format : fiche `CLAUDE.md` ; types de questions : déduits des supports

Format annoncé : QCM et questions ouvertes, 50 points, 2 heures. Barème détaillé et documents autorisés : **À VÉRIFIER**. Les séries d'entraînement 2026 montrent un gabarit à 6 exercices : un bloc VRAI/FAUX à $+1$ et -1, puis des calculs courts où la démarche est exigée. Les types ci-dessous sont une hypothèse de travail tirée des slides, des TP et de ces séries.

Question probable	Où réviser
contraintes critiques d'une application	concept 1 (trois exemples)
définitions IA, ML, DL, génératif ; rationalité	concepts 2 et 3
TinyML, EdgeAI, quantification, élagage	concept 5
définition de O, preuve avec c et N_0	concept 6
pièces, tris, comparaison de croissances	concept 6
supervisé, non supervisé, renforcement	concept 7
les huit défis, cloud ou local	concepts 8 à 10
ratios d'énergie, budget d'une inférence	concept 11
bits d'un format, taille d'un modèle	concept 13
lire du code Python ou NumPy, prédire la sortie	concepts 14 et 15
équations normales, ridge, RMSE à la main	concepts 16 à 18
sigmoïde, frontière, un pas de gradient	concept 19

Question probable, cours 3 à 6	Où réviser
kNN, nombre d'opérations	concept 21
arbre de décision, entropie, gain	concepts 22 et 23
surapprentissage, découpage des données	concepts 24 et 43
matrice de confusion depuis un tableau de seuils	concept 25
ROC, AUC, attribut sans rapport	concept 26
distance adaptée, Mahalanobis	concept 28
k-means, une itération, sens des centres	concept 29
dendrogramme, nombre de clusters, linkage	concept 30
PCA à la main, variance décrite	concepts 33 et 34
cas d'usage : matrice à 3 classes, int8	concept 35
sortie d'un neurone, activations	concepts 37 et 38
nouvelle valeur d'un poids (quiz 06.3)	concepts 40 et 41
vanishing gradient, early stopping	concepts 38, 42 et 43

À retenir

1. QCM : relire chaque option contre la slide ; attention aux options absolues (toujours, jamais) et aux inversions (CPU latence, GPU débit).
2. Question ouverte : définir, citer la slide (exemple chiffré), conclure pour l'embarqué.
3. Calcul : écrire la formule, poser les unités (pJ, Wh, octets), vérifier l'ordre de grandeur.
4. Régression : $\mathbf{X}^\top\mathbf{X}$ et $\mathbf{X}^\top\mathbf{y}$ d'abord, puis l'inverse 2×2 par la formule du déterminant.
5. Gain d'information, k-means, rétropropagation : poser un tableau (branches, distances au carré, neurones), puis calculer ligne par ligne.

Piège

- Oublier la colonne de 1 dans $\mathbf{X}$ (pas d'ordonnée à l'origine).
- Confondre MSE et RMSE, ou calculer la RMSE sur les données d'entraînement.
- Confondre la classe O et le nombre réel d'opérations (pièces, tri).
- Compter les erreurs par `np.sum(pred - y)`.
- Oublier les poids $\frac{p_k+n_k}{p+n}$ dans le reste d'un gain d'information.
- Échanger classe positive et négative, donc sensibilité et spécificité.
- Rétropropagation : facteur 2 oublié (ou ajouté), mauvais signe de l'erreur, poids de sortie déjà mis à jour utilisé pour le gradient caché.

46. Formulaire

À retenir

Note finale	examen +0,2× midterm + quiz ; réussite à 24
Signature	midterm $\geq 20/50$
$T(N) = O(F(N))$	$\exists\, c, N_0 > 0$, $T(N) \leq cF(N)$ pour $N > N_0$
Pièces	force brute $N-1$; tiers $\lceil \log_3 N \rceil$
Tris	bulles $O(N^2)$, mémoire $O(1)$; merge $O(N \log N)$, mémoire $O(N)$
Énergie (pJ)	add. 32b 0,1 ; add. FP32 0,9 ; mult. FP32 3,7 ; mult. 8b 0,2 ; SRAM 5 ; DRAM 640
DRAM	$P = \frac{1}{2} \sum C_i V_i^2 f_i$
Formats	FP32 (1, 8, 23) ; FP16 (1, 5, 10) ; int8 de 0 à 127
Taille modèle	paramètres × octets (4, 2, 1)
Moindres carrés	$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$
Ridge	$\mathbf{w} = (\mathbf{\Phi}^\top \mathbf{\Phi} + \lambda \mathbf{I})^{-1} \mathbf{\Phi}^\top \mathbf{y}$
Inverse 2×2	$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$
RMSE	$\sqrt{\frac{1}{N} \sum (y_i - \hat{y}_i)^2}$
Sigmoïde	$\sigma(a) = 1/(1+e^{-a})$; $\sigma(0) = 0{,}5$; $\sigma' = \sigma(1-\sigma)$
Logistique	$\nabla l = \mathbf{\Phi}^\top (\sigma(\mathbf{\Phi}\mathbf{w}) - \mathbf{y})$; $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla l$
Perte en $\mathbf{w} = 0$	$N \ln 2$ ($\ln 2 \approx 0{,}693$)

À retenir

kNN, k-means	$O(dN)$ par point ; $O(NKJd)$, dont NKd par itération
Entropie	$I = -\sum_j P(j)\log_2 P(j)$; $I(\frac{1}{2},\frac{1}{2}) = 1$; $I(0{,}8;\ 0{,}2) = 0{,}722$
Gain	$I(\frac{p}{p+n},\frac{n}{p+n}) - \sum_k \frac{p_k+n_k}{p+n} I(\frac{p_k}{p_k+n_k},\frac{n_k}{p_k+n_k})$
Instances, fonctions	n attributs binaires : 2^n et 2^{2^n}
Sensibilité, spécificité	$\frac{TP}{TP+FN}$; $\frac{TN}{TN+FP}$; FPR = 1− spécificité
Précision, accuracy	$\frac{TP}{TP+FP}$; $\frac{TP+TN}{\text{total}}$; $F_1 = \frac{2PS}{P+S}$
ROC, AUC	TPR en fonction de FPR ; hasard 0,5 ; idéal 1
Mahalanobis (2D)	$\sqrt{(\Delta x/\sigma_1)^2 + (\Delta y/\sigma_2)^2}$
Yinyang	$ub' = ub + d(b,b')$; $lb' = lb - d(c_k, c'_k)$; sauter si $ub' < lb'$
PCA	$\mathbf{\Sigma} = \frac{1}{N}\mathbf{X}_{(st)}^\top \mathbf{X}_{(st)}$; variance décrite $\sum_{i\le r}\lambda_i / \sum \lambda_i$; $\lambda_i = s_i^2/N$
Valeurs propres 2×2	$\lambda^2 - \operatorname{tr}\lambda + \det = 0$

À retenir

Neurone	$v = \sum_i w_i x_i + b$; $y = f(v)$
Logistique, tanh	$f' = a\,y(1-y)$; $\tanh' = 1 - y^2$; $\sigma' \le 0{,}25$
Rétropropagation	$\Delta w_{ji} = \eta\,\delta_j\,y_i$; sortie $\delta = c\,e\,f'(v)$; caché $\delta_j = f'(v_j)\sum_k \delta_k w_{kj}$; $c = 1$ si $E = \frac{1}{2}e^2$, $c = 2$ si $E = e^2$
Moment	$\Delta w(n) = \alpha\,\Delta w(n-1) + \eta\,\delta\,y$, $0 \le \alpha < 1$
Régularisation	$\gamma\,\text{mse} + (1-\gamma)\,\text{msw}$
Découpage	60 : 20 : 20 (`NN2a`) ; 70 : 15 : 15 (`05_UseCase`)
Anomalie (`05_UseCase`)	$d(\mathbf{x}) = \min_k(\lVert\mathbf{x} - c_k\rVert_2 - r_k)$

Ordres de grandeur	
ChatGPT	$2 \cdot 10^9$ requêtes $\times$ 0,34 Wh $= 0{,}68 \cdot 10^9$ Wh par jour
GPT-4 (entraînement)	62 318 800 kWh, 100 jours, environ 48 fois GPT-3
AlphaGo	1920 CPU, 280 GPU, environ 1 MW, 3000 $ par partie
Cerveau	1,3 kg, 20 W : 1 MW = 50 000 cerveaux
ResNet18 à 152	erreur 10,76 à 6,16 % ; 2,5 jours à 1,5 semaine
VGG16	134,2 M paramètres, 512 Mo
NumPy	
`linspace(a,b,n)`	n points, pas $(b-a)/(n-1)$
`axis=0` / `axis=1`	résultat par colonne / par ligne
`A[:, j]`, `A.T`, `@`	colonne, transposée, produit matriciel